

Learning-to-Optimize via Deep Unfolded Flows

Augustinos D. Saravanos¹ Oswin So¹ H. M. Sabbir Ahmad² Chuchu Fan¹

Abstract

We introduce **FlowOptimizer**, a deep unfolded, flow-based framework for learning iterative optimizers. Motivated by the expressiveness of flow models, we represent each optimization iteration via a velocity field that operates on a population of candidate solutions, i.e., a set of parallel iterates, conditioned on contextual information including their objective values and gradients, as well as population-level statistics. The velocity field is initially trained in a simulation-free manner by matching displacements from source populations to improved target ones obtained through sampling. Subsequently, we unfold the flow model across optimization iterations and fine-tune the entire sequence end-to-end by directly optimizing objective values over a targeted class of problems. Notably, FlowOptimizer is a self-supervised framework whose training relies solely on objective evaluations without requiring knowledge of solutions. We evaluate our approach on a series of tasks from standard non-convex optimization benchmarks to real-world problems from robotics, power grid and supply chain applications. FlowOptimizer consistently outperforms well-established sampling-based/gradient-based traditional optimization and learning-to-optimize methods by orders of magnitude in terms of solution quality. We further highlight its ability to be trained on low-dimensional problems and successfully generalize to substantially higher-dimensional ($\times 10$) ones.

1. Introduction

Solving optimization problems efficiently lies at the core of modern engineering and machine learning applications.

¹Department of Aeronautics and Astronautics, Massachusetts Institute of Technology, Cambridge, MA, USA ²Division of Systems Engineering, Boston University, Boston, MA, USA. Correspondence to: Augustinos D. Saravanos <asaravan@mit.edu>.

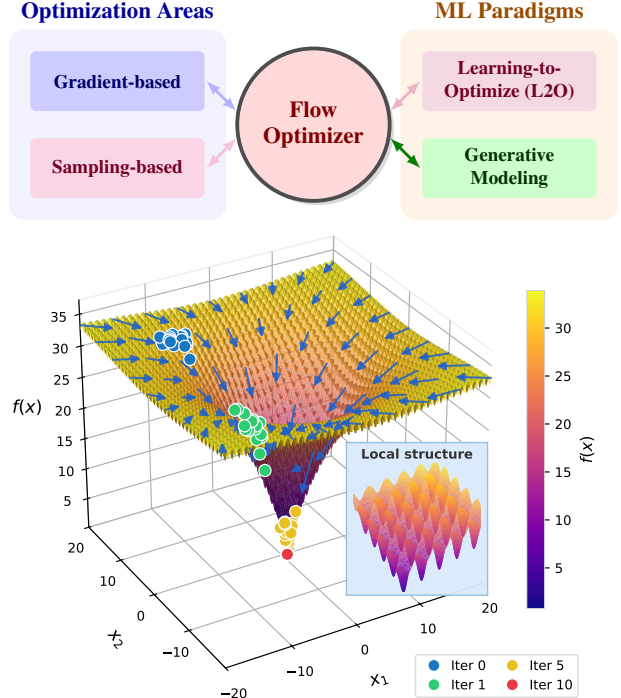


Figure 1. Conceptual overview. *Top:* From an optimization perspective, FlowOptimizer bridges gradient-based and sampling-based methods, while from a machine learning point of view, it lies at the intersection of learning-to-optimize and generative modeling. *Bottom:* FlowOptimizer represents each iteration via a learned velocity field conditioned on context information (objective values, gradients, statistics history, etc.), whose goal is to transport a population of candidate solutions to lower-cost regions. The shown example is the highly non-convex 2D Ackley function with an abundance of local minima. The arrows show the velocity field at iteration 5 w.r.t. the current population/context.

Some of the most fundamental challenges arise from non-convexity leading to numerous local optima, as well as from the increasing scale of the problems, which significantly increases their complexity and computational demands. At the same time, practical relevance demands methods that remain effective under limited computational budgets and generalize well across different problems without the need for extensive manual tuning.

Traditionally, optimization methodologies can be classified into two broad families: gradient-based and sampling-based

techniques. The former class exploits local derivative information to descend toward lower-cost regions (Wright et al., 1999), yet it is prone to getting trapped into poor local minima in non-convex settings. In contrast, sampling-based methods tend to be more robust to non-convexity (Kochenderfer & Wheeler, 2019), but typically face significant scalability challenges in high-dimensional regimes. Although there exists a notable amount of literature on hybrid approaches (Neri & Cotta, 2012; Wierstra et al., 2014), the pursuit of algorithms that effectively integrate gradient information and exploration via sampling remains an open challenge. Most importantly, striking an optimal balance between the two mechanisms is directly tied to the underlying problem geometry. Therefore, traditional optimization schemes with hand-crafted algorithmic choices naturally face difficulties to adapt across diverse landscapes.

Learning-to-optimize (L2O) replaces or augments hand-crafted optimization algorithms by parameterizing update operators with neural networks trained over a target distribution of problems (Chen et al., 2022; Shlezinger et al., 2022; Amos et al., 2023). A prominent L2O paradigm is *deep unfolding*, which maps iterations of an optimization algorithm into layers of a deep neural network (Gregor & LeCun, 2010; Monga et al., 2021). By unrolling a finite number of optimization steps and training end-to-end, unfolded architectures can exploit underlying structure to achieve substantially faster convergence and improved robustness (Sambharya & Stellato, 2024; Saravanos et al., 2025; Oshin et al., 2026). However, most existing L2O and deep unfolding methods parameterize single-point, gradient-based updates. While effective in convex or mildly non-convex settings, these deterministic local updates can become ineffective in complex, multimodal, high-dimensional landscapes.

From a different angle, generative modeling has emerged as a powerful framework for learning complex distributions, with diffusion (Ho et al., 2020; Song et al., 2021) and flow-based models (Chen et al., 2018; Lipman et al., 2023; Liu et al., 2023) gaining widespread attention. While continuous-time diffusion relies on stochastic differential equations (SDEs), flow-based models parameterize continuous-time vector fields governed by ordinary differential equations (ODEs). These flow-based approaches have recently enjoyed particular success because learning straight transport trajectories via flow matching enables stable, simulation-free training (Lipman et al., 2023; Albergo & Vanden-Eijnden, 2023) and significantly faster sampling using numerical ODE integrators compared to stochastic continuous-time diffusion (Liu et al., 2023).

Driven by viewing optimization as seeking a distribution of points whose mass progressively concentrates in low-cost regions of the objective landscape, our key insight is that the expressiveness of flow-based models can be leveraged to

learn population-level optimization dynamics. Rather than learning direct mappings to solutions or hand-designing update rules, we propose to model the optimization process itself as a learned transport of a population of candidate solutions, conditioned on their objective values, gradient information and population-level statistics. This viewpoint naturally enables a principled integration of gradient-driven exploitation and sampling-based exploration within a single, learnable framework that relies on velocity fields to guide the population towards lower-cost regions.

Specifically, our contributions are the following:

- We introduce **FlowOptimizer**, a novel learning-to-optimize framework whose core iteration is defined by a flow-based model that updates a population of candidate solutions conditioned on objective values, gradient information and statistics history. The proposed model can be pre-trained in a simulation-free manner using a flow matching objective, without requiring access to ground-truth optimal solutions.
- We further present a deep unfolded optimization architecture by unrolling the iterations of FlowOptimizer as layers of a deep neural network. This formulation enables end-to-end, self-supervised fine-tuning by directly minimizing a training loss defined in terms of objective evaluations, allowing the framework to adapt its dynamics across optimization iterations.
- Through extensive experimental evaluation, we demonstrate that FlowOptimizer consistently outperforms well-established gradient-based, sampling-based and L2O baselines across a range of highly non-convex optimization benchmarks, as well as real-world applications in robotics, power grids and supply chains. We further show that the dimension-agnostic variation of the proposed approach exhibits strong generalization to significantly higher-dimensional ($\times 10$) settings.

2. Related Work

An overview of related work in learning-to-optimize, generative modeling for optimization and learning-to-sample is provided below. For an extended discussion, we refer the reader to Appendix A.

Learning-to-Optimize and Deep Unfolding. Learning-to-optimize methodologies aim to learn optimization updates by training on problem data (Chen et al., 2022; Shlezinger et al., 2022; Amos et al., 2023). These approaches range from learning direct mappings from problem instances to solutions (Amos et al., 2023; Van Hentenryck, 2025) to more structured methods such as deep unfolding, which unrolls iterative optimization algorithms as sequential

layers of a deep neural network, enabling end-to-end learning (Monga et al., 2021; Zhang et al., 2020). Deep unfolding approaches can range from simply learning algorithm hyperparameters (Khani et al., 2020) to incorporating neural networks within optimization iterations (Samuel et al., 2019; Shlezinger et al., 2020). Recent works have further advanced deep unfolding by incorporating more expressive architectures and learning strategies (Sambharya & Stellato, 2024; Saravanos et al., 2025; Oshin et al., 2026), demonstrating improved performance and generalization across several constrained and large-scale optimization domains.

Generative Modeling for Optimization. Generative models have been gaining increasing attention for addressing optimization and decision-making tasks. Kong et al. (2025) introduced a diffusion-based method for constrained sampling in optimization with unknown constraints. Similarly, Khalafi et al. (2024) presented a method for training diffusion models to satisfy constraints based on the Kullback–Leibler (KL) divergence. Uehara et al. (2024) proposed a hybrid method that finetunes diffusion models by optimizing rewards, while Mazé & Ahmed (2023) presented an effective diffusion-based approach for topology optimization. Other approaches for solving KL-regularized optimization with diffusion models have been presented in (Domingo-Enrich et al., 2025; So et al., 2026). Janner et al. (2022) treated dynamic trajectory optimization as conditional generation with diffusion models, while some notable flow-based approaches can also be found in (Kurtz & Burdick, 2025) for model predictive control and (Idoko & Singh, 2025) for multi-robot trajectory optimization. Finally, Sun & Yang (2023) and Li et al. (2023) presented diffusion-based approaches for obtaining high-quality solutions in combinatorial optimization. A broader discussion on flow-based models as iterative algorithms in probability space is provided in (Xie & Cheng, 2026).

Learning-to-Sample. The problem of drawing samples from an unnormalized target distribution is closely related to optimization (Cheng, 2020). For example, by viewing the unnormalized target density as a Gibbs distribution associated with an objective function and a temperature parameter, sample probability increasingly concentrates around the global energy minimizers as temperature decreases (Varadhan, 1984); a property exploited by simulated annealing (Geman & Geman, 1984). Traditional sampling methods such as Markov Chain Monte Carlo (MCMC) (Geyer, 1992) are widely used, but often suffer from slow mixing in high dimensions or multi-modal landscapes (Łatuszyński et al., 2025). To address these challenges, recent works have explored learning neural components of samplers to accelerate convergence and improve sample quality (Zhang & Chen, 2022; Richter & Berner, 2024), including diffusion-based samplers that approximate the time-reversal of a diffusion

carrying the target density to a Gaussian (Vargas et al., 2023), or that are trained solely from the energy function and its gradient without data or MCMC samples (Akhound-Sadegh et al., 2024). However, many of these methods do not leverage information about the objective at multiple points simultaneously. Particle-based variational inference (Liu & Wang, 2016) offers an alternative by maintaining a population of interacting particles, with neural extensions proposed in (di Langosco et al., 2021).

3. FlowOptimizer: A Flow-Based Model as the Core Optimization Mechanism

3.1. Problem Statement

We consider optimization problems of the form

$$\min_{x \in \mathbb{R}^n} f(x) \quad (1)$$

where $x \in \mathbb{R}^n$ is the decision variable and $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a potentially non-convex objective function to be minimized. We assume access to evaluations of $f(x)$ at arbitrary points x , and, when available, to gradients $\nabla f(x) \in \mathbb{R}^n$. Given a class of objective functions $\mathcal{F}$, our goal is to learn an optimizer that, for previously unseen instances $f \in \mathcal{F}$, achieves low objective values, as measured by the best candidate solution found, within a limited computational budget.

3.2. Flow Model as the Core Optimization Iteration

From a sampling point of view, optimization can be interpreted as seeking to sample from low-value regions of the cost landscape. Through this lens, optimizing a class of objective functions can be viewed as a generative modeling task that seeks a transport map from a simple initial distribution p^0 over candidate points to a target distribution p^* whose mass concentrates on low-value regions of the objectives $f \in \mathcal{F}$, e.g., a Gibbs distribution $p^*(x) \propto \exp(-f(x)/\lambda)$, with small temperature $\lambda > 0$. Nevertheless, directly employing diffusion or flow-based models to learn solutions in an end-to-end manner might be restrictive, as such a one-shot transformation must capture the entire optimization process at once. Another major challenge is that the target distribution is only implicitly defined by the objectives $f \in \mathcal{F}$, and ground-truth solutions $x^* = \arg \min_x f(x)$ are likely to be unavailable in non-convex or high-dimensional settings, which fundamentally limits the applicability of supervised learning approaches for generative models.

In parallel, most algorithms within sampling-based optimization rely on iteratively updating a parametric distribution based on information coming from the current points population. Concretely, at iteration k , a population $\{x_j^k\}_{j=1}^P$, where P is the population size, is sampled from a parametric distribution q_{θ^k} , whose parameters are iteratively updated

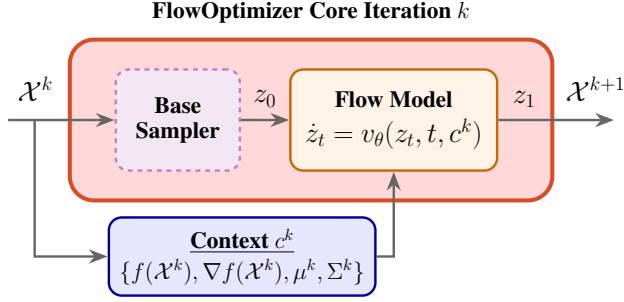


Figure 2. The core optimization iteration of FlowOptimizer. At iteration k , the current population $\mathcal{X}^k$ is optionally processed by a base sampler and then evolved through a flow model. The flow integrates population-level dynamics conditioned on contextual information c^k , producing the updated population $\mathcal{X}^{k+1}$.

based on information extracted from the current population. Such algorithms, however, are often limited by the restricted expressiveness of their parametric families (typically Gaussian) and by update rules that exploit only a small portion of the available contextual information. Motivated by these complementary perspectives, our aim is to *learn* a flow-based model that acts as an *iterative optimizer* operating on a *population* of candidate solutions.

Population-Level Update via Flows. Our goal is to learn a model $\mathcal{T}(\cdot)$ that transforms populations of points $\mathcal{X}^k = \{x_j^k\}_{j=1}^P$ to lower-cost ones $\mathcal{X}^{k+1} = \{x_j^{k+1}\}_{j=1}^P$, i.e.,

$$\mathcal{X}^{k+1} = \mathcal{T}(\mathcal{X}^k, c^k), \quad (2)$$

where $c^k \in \mathcal{C}$ denotes contextual information available at iteration k . We parameterize the transformation $\mathcal{T}(\cdot)$ as the flow induced by a continuous-time velocity field defined over the population. Specifically, we treat the population as a single state $z_t \in \mathbb{R}^{P \times n}$ evolving according to

$$\dot{z}_t = v_\theta(z_t, t, c^k), \quad z_{t=0} = \mathcal{X}^k, \quad (3)$$

where $v_\theta : \mathbb{R}^{P \times n} \times [0, 1] \times \mathcal{C} \rightarrow \mathbb{R}^{P \times n}$ is a learnable, context-conditioned velocity field parameterized by θ . During inference, the population update is obtained by integrating and assigning $\mathcal{X}^{k+1} = z_{t=1}$. Importantly, the velocity field acts jointly on the entire population, thus benefiting from capturing population-level interactions.

Context Information. In its most basic form, the context variable c^k includes the objective values $f(\mathcal{X}^k) = \{f(x_j^k)\}_{j=1}^P$ and gradients $\nabla f(\mathcal{X}^k) = \{\nabla f(x_j^k)\}_{j=1}^P$, as well as distributional statistics such as the empirical mean $\mu^k = \mathbb{E}_{x \in \mathcal{X}^k}[x]$ and covariance $\Sigma^k = \mathbb{E}_{x \in \mathcal{X}^k}[(x - \mu^k)(x - \mu^k)^\top]$ of the current population $\mathcal{X}^k$. The velocity field can be conditioned on additional informative signals such as the population-average objective $\bar{f}^k = \mathbb{E}_{x \in \mathcal{X}^k}[f(x)]$ and gradient $\bar{g}^k = \mathbb{E}_{x \in \mathcal{X}^k}[\nabla f(x)]$, and the weighted covariance

$\text{Cov}(\mathcal{X}^k, f(\mathcal{X}^k)) = \mathbb{E}_{x \in \mathcal{X}^k}[(x - \mu^k)(f(x) - \bar{f}^k)]$ which provides a zero-order estimate of the local slope of the landscape. The objective values may further be encoded via rank or z-score transformations to improve robustness to the scale of f . Note that all of the above quantities are computed from the current population $\mathcal{X}^k$ alone. As shown later, once the optimizer is unfolded across iterations, however, the context can additionally aggregate information from past iterations.

Permutation Equivariance via Attention. Since the population $\mathcal{X}^k$ is an unordered set, the learned velocity field v_θ must be permutation-equivariant with respect to the population members. Formally, for any permutation π acting on the population indices, we require

$$v_\theta(\pi z_t, t, \pi c_{\text{loc}}^k, c_{\text{glob}}^k) = \pi v_\theta(z_t, t, c_{\text{loc}}^k, c_{\text{glob}}^k), \quad (4)$$

where the context c^k is decomposed into the (local) member-level information part $c_{\text{loc}}^k = \{c_j^k\}_{j=1}^P$, with $c_j^k = \{f(x_j^k), \nabla f(x_j^k)\}$, and the (global) population-level part $c_{\text{glob}}^k = \{\bar{f}^k, \bar{g}^k, \dots\}$. To achieve permutation equivariance, we parameterize v_θ using a self-attention architecture that operates jointly on the population state and the local context. Letting $\phi_z(\cdot)$ and $\phi_c(\cdot)$ denote shared embedding maps, we form joint per-member representations

$$h_j = [\phi_z(z_t^{(j)}), \phi_c(c_j^k)], \quad j = 1, \dots, P, \quad (5)$$

and apply a permutation-equivariant self-attention operator

$$(\tilde{h}_1, \dots, \tilde{h}_P) = \text{SelfAttention}(h_1, \dots, h_P), \quad (6)$$

which enables information exchange across the population while preserving equivariance. The global features c_{glob}^k , along with the flow time t , are incorporated through shared inputs across all population members. The resulting architecture yields a permutation equivariant velocity field, while remaining expressive enough to capture both inter-member interactions and population-level information. We further emphasize the flexibility of the model to use different population sizes during training and deployment.

Base Sampler. An optional base sampling operator $\mathcal{S}(\cdot)$ may be applied to the current population $\mathcal{X}^k$ to generate the initial flow state $z_{t=0}$, i.e., $z_{t=0} = \mathcal{S}(\mathcal{X}^k)$, instead of directly setting $z_{t=0} = \mathcal{X}^k$. When a base sampler is used, the original population $\mathcal{X}^k$ is appended to the context information c^k . A straightforward choice is a Gaussian sampler, where new candidates are drawn from a normal distribution fitted to the best points of the current population, in the spirit of classical sampling-based methods such as the Cross-Entropy Method (CEM) (De Boer et al., 2005). Crucially, in contrast to such methods, the Gaussian distribution is used solely as a source distribution, while the subsequent population transformation is driven by the expressive learned

flow model. Empirically, we observe that in some cases incorporating a base sampler can stabilize the optimization process by inducing a more controlled exploration around the current population. However, incorporating such simple parametric samplers may also restrict population diversity in highly multimodal landscapes. By keeping the base sampler optional, our framework can leverage its stabilizing effect if needed, while relying on the flow model to capture richer population transformations.

3.3. Dimension-Agnostic Variant

To enable the same model to be applicable to different problem dimensions as well, we present a dimension-agnostic parameterization of the velocity field. Given a population state $z_t = \mathcal{X}^k \in \mathbb{R}^{P \times n}$, objectives $f(\mathcal{X}^k) \in \mathbb{R}^P$ and gradients $\nabla f(\mathcal{X}^k) \in \mathbb{R}^{P \times n}$, we construct fixed-size *per-member tokens* by pooling statistics across dimensions, i.e.,

$$s_{t,j} = [\text{Stat}(z_{t,j}), f(x_j^k), \text{Stat}(\nabla f(x_j^k)), t] \in \mathbb{R}^m, \quad (7)$$

for each member $j \in \{1, \dots, P\}$. The statistics pooling operator is given by $\text{Stat}(\cdot) = [\text{Mean}(\cdot), \text{Std}(\cdot), \text{Max}(\cdot)]$. These per-member tokens are then mapped through a shared embedding network to obtain $h_j \in \mathbb{R}^H$. We then apply self-attention over $\{h_j\}_{j=1}^P$ to capture population-level interactions while remaining agnostic to problem dimension n . To generate velocities, we use a *shared per-dimension decoder* applied independently across coordinates:

$$v_{j,i}^{\text{loc}} = \psi_\theta([z_{t,j}]_i, [\nabla f(x_j^k)]_i, h_j, t), \quad i = 1, \dots, n, \quad (8)$$

which enforces parameter sharing across dimensions and yields a coordinate-wise equivariant update. This design yields a velocity field $v_\theta(z_t, t, c^k) \in \mathbb{R}^{P \times n}$ whose parameter count does not scale with n , facilitating generalization across dimensions.

3.4. Connections to Classical and Learned Optimizers

The representation employed by FlowOptimizer is sufficiently expressive to recover several classical and learned optimization paradigms as special cases. When a Gaussian base sampler is used, the resulting population dynamics closely resemble sampling-based methods such as CEM and related evolutionary strategies. In contrast, when no base sampler is employed and the population size is reduced to one, the update rule recovers a deterministic, gradient-driven iteration, closely aligning with classical first-order methods as well as learning-to-optimize approaches that learn parametric update rules. When the population size is greater than one but no base sampler is used, the deterministic interactions among candidate solutions bear resemblance to particle swarm optimization (PSO), where particles update their states by aggregating information from their local history and the collective population. Beyond these limiting

Algorithm 1 Pre-training phase via flow matching

```

1: Input: Model  $v_\theta$ , objective set  $\{f_i\}_{i=1}^N \subset \mathcal{F}$ , population size  $P$ , improvement operator  $\text{Impr}(\cdot, f)$ , improvement sequence length  $G$ , pair offset  $\Delta$ , epochs  $E$ , minibatch size  $M$ 
2: // Construct source-target couplings
3: Initialize dataset  $\mathcal{D} \leftarrow \emptyset$ 
4: for  $i = 1$  to  $N$  do
5:   Sample  $\mathcal{X}^0 \sim p_0$ 
6:   Run  $\mathcal{X}^{k+1} \leftarrow \text{Impr}(\mathcal{X}^k, f_i)$  for  $G$  iterations
7:   Assign pairs  $\{(\mathcal{X}, \mathcal{X}^+) \leftarrow (\mathcal{X}^k, \mathcal{X}^{k+\Delta})\}_{k=0}^{G-\Delta}$ 
8:   Get context  $c \leftarrow \{f_i(\mathcal{X}), \nabla f_i(\mathcal{X}), \dots\}$ 
9:    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathcal{X}, \mathcal{X}^+), c\}$ 
10: end for
11: // Flow matching
12: for epoch = 1 to  $E$  do
13:   for each minibatch  $\{(\mathcal{X}_b, \mathcal{X}_b^+, c_b)\}_{b=1}^M \subset \mathcal{D}$  do
14:     Set  $z_0^b \leftarrow \mathcal{X}_b$ ,  $z_1^b \leftarrow \mathcal{X}_b^+$  (each in  $\mathbb{R}^{P \times n}$ )
15:     Sample  $t_b \sim \mathcal{U}[0, 1]$ 
16:     Set  $z_{t_b}^b \leftarrow (1 - t_b) z_0^b + t_b z_1^b$ 
17:      $\mathcal{L}_{\text{FM}}(\theta) \leftarrow \frac{1}{M} \sum_{b=1}^M \|v_\theta(z_{t_b}^b, t_b, c_b) - (z_1^b - z_0^b)\|_2^2$ 
18:     Update  $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{FM}}(\theta)$ 
19:   end for
20: end for
    
```

cases, FlowOptimizer enables rich interactions among multiple candidate solutions through learned population-level dynamics, allowing it to effectively blend the complementary strengths of sampling-based global exploration and gradient-based local refinement.

3.5. Pre-Training via Flow Matching

We initially train a single model instance (i.e., one iteration) $\mathcal{T}(\cdot)$ of the flow optimizer using the flow matching loss due to its training efficiency. In particular, we train the velocity field v_θ to match displacements from randomly drawn source population points $\mathcal{X}$ to improved ones $\mathcal{X}^+ = \text{Impr}(\mathcal{X}, f)$ in terms of their objective values. The improved population can be obtained by sampling initial populations and then further sampling with an increased variance or running a reliable optimizer. Given pairs (z_0, z_1) with $z_0 \in \mathcal{X}$ and $z_1 \in \mathcal{X}^+$, we define an interpolation

$$z_t = (1 - t)z_0 + tz_1, \quad t \sim \mathcal{U}[0, 1], \quad (9)$$

and train the velocity field to predict the displacement $\Delta z = z_1 - z_0$ along this path, conditioned on the context information. The flow matching training loss is then

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, z_0, z_1} \|v_\theta(z_t, t, c) - \Delta z\|_2^2 \quad (10)$$

where the expectation is taken over source-target pairs $(z_0, z_1) \in \mathcal{X} \times \mathcal{X}^+$, and interpolation times $t \sim \mathcal{U}[0, 1]$.

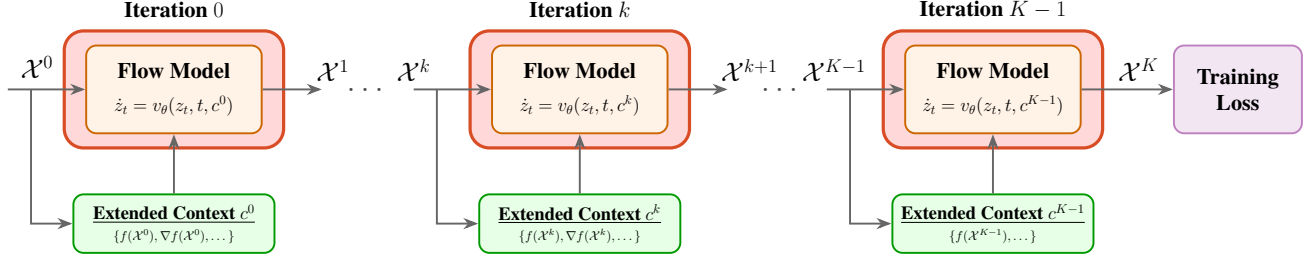


Figure 3. The full deep unfolded FlowOptimizer architecture. By unrolling the iterations of FlowOptimizer as sequential layers in a deep neural network, we obtain the final architecture, which is trained in an end-to-end fashion via a self-supervised training loss.

This pre-training phase yields a single learned update $\mathcal{T}(\cdot)$ that captures a generic improvement direction across a class of objectives, without requiring access to ground-truth solutions. The pre-training process is detailed in Alg. 1.

4. Deep Unfolded FlowOptimizer

The full proposed architecture emerges by unfolding FlowOptimizer and training in an end-to-end manner. In Section 4.1, we introduce the deep unfolded architecture which unrolls the iterations of FlowOptimizer as sequential layers. In Section 4.2, we then describe an end-to-end fine-tuning procedure in which all unfolded layers are jointly optimized by directly minimizing the objective values induced by the population iterates.

4.1. Unfolding the Flow Model as an Iterative Optimizer

We unfold FlowOptimizer for a prescribed number of iterations K , treating each iteration as a sequential layer in our architecture (Fig. 3). To accommodate the distinct roles played by different stages of the optimization process, we allow for each flow model to vary across iterations, resulting in iteration-specific models $\mathcal{T}^k$, $k = 0, \dots, K-1$, with corresponding velocity fields $v_{\theta^k}(z_t^k, t, c^k)$. Each iteration is initialized with the same pre-trained model obtained via flow matching. Furthermore, to improve steady-state behavior, we share parameters across the final K_{final} iterations by repeating the same velocity model across the iterations $K - K_{\text{final}}, \dots, K-1$. This design both stabilizes the terminal phase of the optimization and enables the unfolded network to be executed for additional iterations at inference time, if desired, without introducing new parameters.

4.2. End-to-End Fine-Tuning

Subsequently, we fine-tune the deep unfolded model using a training loss that directly minimizes the objective values $f(\mathcal{X}^k)$. To obtain each new iterate $\mathcal{X}^{k+1}$, we perform Euler integration on the velocity field ODEs,

$$z_{t+\Delta t}^k = z_t^k + v_{\theta}(z_t, t, c^k)\Delta t \quad (11)$$

Algorithm 2 Fine-tuning phase via deep unfolding

- 1: **Input:** Pre-trained model v_{θ} , objective class $\mathcal{F}$, population size P , unrolling depth K , epochs E , minibatch size M , loss weights $\alpha, \{w_k\}_{k=1}^K$
- 2: **for** epoch = 1 to E **do**
- 3: **for** each minibatch $\{f_b\}_{b=1}^M \subset \{f_i\}_{i=1}^N$ **do**
- 4: Select objective function $f_b \in \mathcal{F}$
- 5: Sample $\mathcal{X}^0 \sim p_0$
- 6: **for** $k = 0$ to $K-1$ **do**
- 7: Get context $c_k \leftarrow \{f_b(\mathcal{X}^k), \nabla f_b(\mathcal{X}^k), \dots\}$
- 8: Set $z_0 \leftarrow \mathcal{X}^k$
- 9: $z_1 \leftarrow \text{IntegrateODE}(\dot{z} = v_{\theta}(z_t, t, c_k), z_0)$
- 10: Set $\mathcal{X}^{k+1} \leftarrow z_1$
- 11: Compute current iteration loss $\ell_{k+1}(\theta) \leftarrow \alpha \text{Best}(f_b(\mathcal{X}_{k+1})) + (1 - \alpha) \text{Mean}(f_b(\mathcal{X}_{k+1}))$
- 12: **end for**
- 13: Compute total loss $\mathcal{L}_{\text{FT}}(\theta) \leftarrow \sum_{k=1}^K w_k \ell_k(\theta)$
- 14: Update $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\text{FT}}(\theta)$
- 15: **end for**
- 16: **end for**

where Δt denotes the discretization step.

Then for training, we define the per-iteration loss

$$\ell_k(\mathcal{X}^k) = \alpha \text{Best}(f(\mathcal{X}_k)) + (1 - \alpha) \bar{f}^k \quad (12)$$

where $\text{Best}(f(\mathcal{X}_k)) := \text{SoftMin}_{j=1, \dots, P} f(x_j^k)$ is an approximation of the best objective achieved within $\mathcal{X}^k$ and $\alpha \in [0, 1]$ is a balancing parameter between mean and best objective within the population. Then, the end-to-end training loss is given by

$$\mathcal{L}_{\text{FT}} = \sum_{k=1}^K w_k \ell_k(\mathcal{X}^k) \quad (13)$$

where the weights w_k assign each iteration k a specific amount of importance. The fine-tuning process is detailed in Alg. 2.

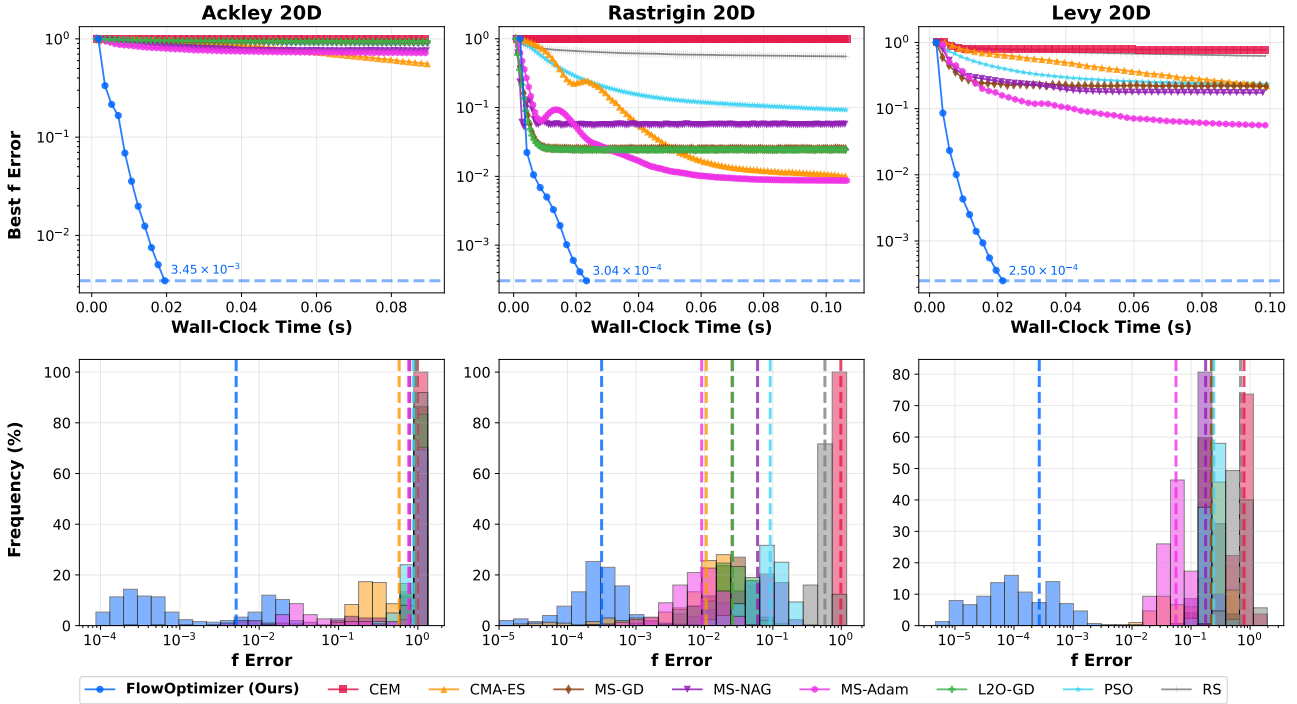


Figure 4. **Standard Optimization Benchmarks.** Comparison of metric $|\text{Best}(f(\mathcal{X}^k)) - f^*| / |\text{Best}(f(\mathcal{X}^0)) - f^*|$ between all methods on three standard optimization benchmarks (Ackley 20D, Rastrigin 20D and Levy 20D). The top row reports the average across all evaluation problem instances. The bottom row reports the distribution of final errors. The proposed FlowOptimizer consistently outperforms all baselines. The baseline methods are allowed to run for $\times 5$ more wall-clock time.

5. Experiments

We demonstrate the performance of **FlowOptimizer** on a diverse set of problems, ranging from standard optimization benchmarks to real-world applications. Section 5.1 summarizes several sampling-based/gradient-based optimization or L2O methodologies used as baselines. In Section 5.2 we compare the methods on various standard optimization benchmarks, while in Section 5.3, we extend this comparison on real-world robotics, power grid and supply chain problems. In Section 5.4, we demonstrate that the proposed dimension-agnostic variant can be trained on low-dimensional problems and then successfully deployed on substantially higher-dimensional ones. Finally, an overall discussion and potential limitations are provided in Section 5.5. For all methods, we use a population of size $P = 20$. The full experimental details are provided in Appendix B.

5.1. Baselines

We compare **FlowOptimizer** against sampling-based methods including Random Search (RS), Cross Entropy Method (CEM) (De Boer et al., 2005), Covariance Matrix Adaptation Evolution Strategy (CMA-ES) (Hansen & Ostermeier, 2001), and Particle Swarm Optimization

(PSO) (Kennedy & Eberhart, 1995), gradient-based techniques with multi-start (MS) randomization (Zhigljavsky, 2025) including MS Gradient Descent (MS-GD), MS Nesterov Accelerated GD (MS-NAG), and MS Adam (MS-Adam), as well as a multi-start variation of Learning-to-Optimize Gradient Descent (L2O-GD) (Sambharya & Stellato, 2024). A full description of all baselines is given in Appendix B.4.

5.2. Standard Optimization Benchmarks

We start with evaluating the performance of all methods on a series of popular non-convex optimization benchmarks. In particular, the objective functions we consider are the Ackley, Rastrigin and Levy functions, all of which present highly non-convex and multimodal cost landscapes that are especially challenging for both sampling-based and gradient-based methods. The functions are fully detailed in Appendix B.5. For all problems, we consider a dimension $n = 20$.

Figure 4 compares convergence speed and final solution quality across all methods on the three benchmarks. The top row reports the average best objective error over wall-clock time, normalized by the initial error, while the bottom row shows the distribution of final errors across prob-

Table 1. **Real-World Optimization Problems.** Average Ratio $\text{Best}(f(\mathcal{X}^K))/f(\mu^0)$ across different methods and problem classes.

Problem Class	Dim. n	FlowOpt (ours)	RS	CEM	CMA-ES	MS-GD	MS-NAG	MS-Adam	L2O-GD	PSO
Robotic Arm	10	0.003	0.18	0.09	0.008	0.014	0.021	0.009	0.012	0.005
	30	0.007	0.31	0.18	0.04	0.18	0.17	0.07	0.12	0.011
Power Grid	20	5e-4	0.53	0.14	0.03	0.05	0.03	0.007	0.03	0.02
	50	1.2e-3	0.83	0.31	0.11	0.23	0.17	0.05	0.08	0.09
Supply Chain	40	0.007	0.47	0.09	0.05	0.07	0.03	0.02	0.013	0.02
	80	0.013	0.68	0.14	0.06	0.09	0.03	0.02	0.027	0.03

lem instances. **FlowOptimizer** consistently achieves the lowest objective error across all three benchmarks, converging rapidly to high-quality solutions within its allowed 10-iteration computational budget. In particular, **FlowOptimizer** reaches final errors orders of magnitude below those of all baselines, despite the baselines being allowed $\times 5$ more wall-clock time. The bottom row histograms further highlight that **FlowOptimizer** not only attains lower errors on average, but does so with markedly tighter distributions, indicating greater reliability across random initializations and problem instances. In contrast, **CEM** and **RS** stagnate near the initial error on all benchmarks, reflecting their poor scalability to high-dimensional multimodal landscapes. **CMA-ES** shows moderate progress on Rastrigin but fails to consistently improve on Ackley and Levy. Gradient-based multi-start methods (**MS-GD**, **MS-NAG**, **MS-Adam**) and **PSO** exhibit high variance and tend to concentrate probability mass at suboptimal error levels, suggesting susceptibility to local minima. Finally, **L2O-GD**, despite leveraging learned update rules, fails to match **FlowOptimizer**, highlighting that learning individual GD steps is insufficient without explicit population-level coordination and a richer representation.

5.3. Real-World Optimization Problems

Subsequently, we evaluate our method on real-world robotics, power grid and supply chain problems. Table 1 reports the average ratio $\text{Best}(f(\mathcal{X}^K))/f(\mu^0)$ across all methods, where lower values indicate better optimization performance relative to the initial solution. Several consistent trends emerge across problem classes and dimensions. Again, the total number of iterations is set to $K = 10$ for **FlowOptimizer**, while the baselines are given as many iterations as needed to match the same computational time.

Robotic Arm Inverse Kinematics. We start with addressing an inverse kinematics task where the arm also needs to avoid obstacles. **FlowOptimizer** outperforms all other methods at low dimension ($n = 10$), with **PSO**, **CMA-ES** and **MS-Adam** being the strongest baselines. Notably, as the dimension increases to $n = 30$, both sampling-based and gradient-based baselines exhibit noticeable degra-

dation with **PSO** remaining the most competitive baseline. **FlowOptimizer** maintains a substantially lower error indicating improved scalability and robustness to increasing redundancy and obstacle-induced non-convexity.

Power Grid Control. Next, we consider the economic load dispatch problem (Sinha et al., 2003), which presents a highly multi-modal landscape due to valve-point effects. Overall, **FlowOptimizer** achieves orders-of-magnitude improvements against all baselines, especially as dimension n increases from 20 to 50. Among the baselines, **MS-Adam** achieves the strongest performance in both cases with **PSO** and **L2O-GD** following. The observed performance gaps compared to **FlowOptimizer** highlight the benefit of integrating gradient information with population-level transport dynamics in highly rugged landscapes.

Supply Chain Optimization. Finally, we study the multi-source Weber problem (Brimberg et al., 2008), where the presence of combinatorial symmetries and clustering ambiguities leads to multiple near-optimal basins. Again, **FlowOptimizer** consistently achieves the lowest error across both $n = 40$ and $n = 80$, suggesting that the learned flow dynamics search effectively in symmetric, non-convex landscapes. At both dimensions, **L2O-GD**, **MS-Adam**, **PSO** and **MS-NAG** remain competitive among the baselines.

Overall Results. Across all real-world problems, **FlowOptimizer** consistently outperforms all sampling/gradient-based classical optimization methods and L2O baselines, with performance gaps that typically widen as dimension increases. These results indicate that learning population-level optimization dynamics enables improved scalability compared to relying on hand-crafted sampling rules and gradient-based updates.

5.4. Generalization to Higher-Dimensional Problems

A well-known limitation of L2O methods is generalizing beyond the problem dimensions seen during training. We evaluate the dimension-agnostic variant of **FlowOptimizer** trained exclusively on 20-dimensional problems and deployed without retraining on 200-dimensional instances of

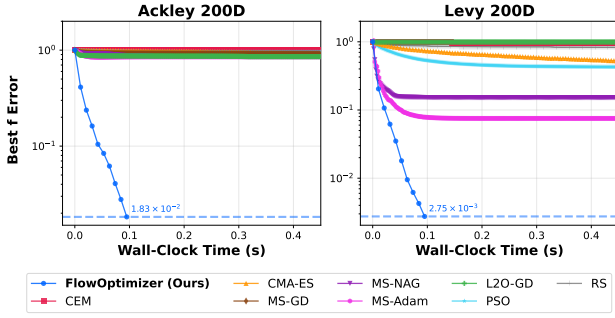


Figure 5. Applying FlowOptimizer (dimension-agnostic variant) to higher-dimensional problems. Models trained on 20D problems are successfully deployed on 200D problems, where the performance of competing baselines degrades significantly.

highly non-convex benchmarks, including Ackley and Levy functions. As shown in Fig. 5, FlowOptimizer retains good performance despite the tenfold increase in dimensionality. In contrast, gradient-based, sampling-based and L2O baselines exhibit severe performance degradation and often fail to make meaningful progress within the same wall-clock budget. These experiments highlight that the proposed flow-based population dynamics enable strong generalization without dimension-specific tuning always being necessary.

5.5. Discussion

What makes FlowOptimizer a stronger method? The substantial empirical gains of FlowOptimizer can be attributed to three key factors.

1. **Expressiveness of flow-based models:** Flow-based models provide a highly expressive parameterization of optimization updates, enabling the learned dynamics to represent complex transport maps that go well beyond fixed hand-designed update rules. This allows FlowOptimizer to adapt its behavior across different regions of the objective landscape and across iterations.
2. **Learning-to-optimize from problem geometry:** By training directly on optimization trajectories, FlowOptimizer learns recurrent patterns induced by the geometry of the underlying problem class, such as curvature, multi-modality, and scale, which are difficult to capture with generic optimizers. This enables the method to internalize when to favor exploration versus refinement in a data-driven manner.
3. **Learning population-level interactions:** Unlike pointwise update rules, FlowOptimizer operates on a population of candidate solutions and explicitly learns interactions among its members. This allows the population to coordinate exploration of the landscape, im-

plicitly sharing information about promising regions, leading to more efficient behavior.

Limitations. One limitation of the proposed L2O architecture is the need to integrate the velocity field during the fine-tuning phase, which can lead to a long computational graph. While in all our experiments we employ a lightweight implementation based on Euler integration with only 5 integration steps, a promising direction for future work is to explore distillation schemes that would compress the integrated dynamics into a shallow surrogate model. Another limitation is that, in its current formulation, the proposed framework is designed for unconstrained optimization and can therefore incorporate constraints only indirectly via penalty or barrier terms. Extending flow-based L2O models to handle constraints, e.g., through projection operators, implicit layers, or integration with constrained optimization solvers, remains an important direction for future research.

6. Conclusion

We introduced FlowOptimizer, a learning-to-optimize framework that models optimization as an iterative, population-level transport process driven by a learned flow. By parameterizing each optimization step as a context-conditioned velocity field and training it through a combination of flow matching pre-training and deep unfolding with objective-driven fine-tuning, FlowOptimizer effectively integrates exploration and exploitation within a unified, self-supervised architecture. Across a wide range of non-convex benchmarks and real-world problems, the proposed method consistently outperforms classical sampling-based, gradient-based and learning-to-optimize baselines, while demonstrating strong generalization to higher-dimensional problems.

Future Directions. Several promising directions emerge from this work. Extending flow-based L2O architectures to constrained optimization in a principled manner, e.g., via projection operators or integration with constrained solvers, remains an important challenge. From a computational viewpoint, reducing the overhead induced by ODE integration through distillation could further improve efficiency.

Acknowledgements

This work was partly supported by the National Science Foundation (NSF) CAREER Award #CCF-2238030.

Impact Statement

This work introduces FlowOptimizer, a learning-to-optimize framework that leverages flow-based models and deep unfolding to construct data-driven optimization algorithms. By enabling scalable and amortized optimization for complex

and high-dimensional problems, the proposed approach has the potential to reduce computational and engineering effort across a range of machine learning and control applications. As with other learned optimization methods, performance may degrade outside the training distribution, and caution is required in safety-critical settings where robustness and reliability are essential. This work does not involve sensitive data or pose direct societal risks and is intended to advance principled research on reliable learning-based optimization.

References

- Akhound-Sadeh, T., Rector-Brooks, J., Bose, J., Mittal, S., Lemos, P., Liu, C.-H., Sendera, M., Ravanbakhsh, S., Gidel, G., Bengio, Y., Malkin, N., and Tong, A. Iterated denoising energy matching for sampling from boltzmann densities. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 760–786. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/akhound-sadeh24a.html>.
- Albergo, M. S. and Vanden-Eijnden, E. Building normalizing flows with stochastic interpolants. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=li7qeBbCR1t>.
- Amos, B. and Kolter, J. Z. OptNet: Differentiable optimization as a layer in neural networks. In Precup, D. and Teh, Y. W. (eds.), *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 136–145. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/amos17a.html>.
- Amos, B. et al. Tutorial on amortized optimization. *Foundations and Trends® in Machine Learning*, 16(5):592–732, 2023.
- Banert, S., Rudzusika, J., Öktem, O., and Adler, J. Accelerated forward-backward optimization using deep learning. *SIAM Journal on Optimization*, 34(2):1236–1263, 2024.
- Bottou, L., Curtis, F. E., and Nocedal, J. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018.
- Boyd, S. and Vandenberghe, L. *Convex optimization*. Cambridge university press, 2004.
- Brimberg, J., Hansen, P., Mladenovic, N., and Salhi, S. A survey of solution methods for the continuous location-allocation problem. *International Journal of Operations Research*, 5(1):1–12, 2008.
- Chen, J., Ma, Z., Guo, H., Ma, Y., Zhang, J., and Gong, Y.-J. Symbol: Generating flexible black-box optimizers through symbolic equation learning. *arXiv preprint arXiv:2402.02355*, 2024.
- Chen, R. T., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.
- Chen, T., Chen, X., Chen, W., Heaton, H., Liu, J., Wang, Z., and Yin, W. Learning to optimize: A primer and a benchmark. *Journal of Machine Learning Research*, 23 (189):1–59, 2022.
- Cheng, X. *The Interplay between Sampling and Optimization*. PhD thesis, University of California, Berkeley, 2020.
- Clerc, M. and Kennedy, J. The particle swarm-explosion, stability, and convergence in a multidimensional complex space. *IEEE transactions on Evolutionary Computation*, 6(1):58–73, 2002.
- De Boer, P.-T., Kroese, D. P., Mannor, S., and Rubinstein, R. Y. A tutorial on the cross-entropy method. *Annals of operations research*, 134(1):19–67, 2005.
- di Langosco, L. L., Fortuin, V., and Strathmann, H. Neural variational gradient descent. *arXiv preprint arXiv:2107.10731*, 2021.
- Domingo-Enrich, C., Drozdal, M., Karrer, B., and Chen, R. T. Q. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=xQBRrtQM8u>.
- Donti, P., Amos, B., and Kolter, J. Z. Task-based end-to-end model learning in stochastic optimization. *Advances in neural information processing systems*, 30, 2017.
- Donti, P. L., Rolnick, D., and Kolter, J. Z. DC3: A learning method for optimization with hard constraints. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=V1ZHVxJ6dSS>.
- Duchi, J., Hazan, E., and Singer, Y. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.
- Fioretto, F., Mak, T. W., and Van Hentenryck, P. Predicting ac optimal power flows: Combining deep learning and lagrangian dual methods. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 630–637, 2020.

- Geman, S. and Geman, D. Stochastic relaxation, gibbs distributions, and the bayesian restoration of images. *IEEE Transactions on pattern analysis and machine intelligence*, (6):721–741, 1984.
- Geyer, C. J. Practical markov chain monte carlo. *Statistical science*, pp. 473–483, 1992.
- Gregor, K. and LeCun, Y. Learning fast approximations of sparse coding. In *Proceedings of the 27th international conference on international conference on machine learning*, pp. 399–406, 2010.
- Hansen, N. The cma evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.
- Hansen, N. and Ostermeier, A. Completely derandomized self-adaptation in evolution strategies. *Evolutionary computation*, 9(2):159–195, 2001.
- Heaton, H., Chen, X., Wang, Z., and Yin, W. Safeguarded learned convex optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 7848–7855, 2023.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Idoko, S. and Singh, A. K. Flow-opt: Scalable centralized multi-robot trajectory optimization with flow matching and differentiable optimization. *arXiv preprint arXiv:2510.09204*, 2025.
- Jacobson, D. H. and Mayne, D. Q. *Differential dynamic programming*. Number 24. Elsevier Publishing Company, 1970.
- Janner, M., Du, Y., Tenenbaum, J., and Levine, S. Planning with diffusion for flexible behavior synthesis. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 9902–9915. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/janner22a.html>.
- Kennedy, J. and Eberhart, R. Particle swarm optimization. In *Proceedings of ICNN’95-international conference on neural networks*, volume 4, pp. 1942–1948. IEEE, 1995.
- Khalafi, S., Ding, D., and Ribeiro, A. Constrained diffusion models via dual training. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=Es2Ey2tGmM>.
- Khani, M., Alizadeh, M., Hoydis, J., and Fleming, P. Adaptive neural signal detection for massive mimo. *IEEE Transactions on Wireless Communications*, 19(8):5635–5648, 2020. doi: 10.1109/TWC.2020.2996144.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Kochenderfer, M. J. and Wheeler, T. A. *Algorithms for Optimization*. MIT Press, 2019. URL <https://algorithmsbook.com/optimization/files/optimization.pdf>.
- Kong, L., Du, Y., Mu, W., Neklyudov, K., Bortoli, V. D., Wu, D., Wang, H., Ferber, A. M., Ma, Y., Gomes, C. P., and Zhang, C. Diffusion models as constrained samplers for optimization with unknown constraints. In *The 28th International Conference on Artificial Intelligence and Statistics*, 2025. URL <https://openreview.net/forum?id=dEwvHEHxPc>.
- Kurtz, V. and Burdick, J. W. Generative predictive control: Flow matching policies for dynamic and difficult-to-demonstrate tasks. *arXiv preprint arXiv:2502.13406*, 2025.
- Łatuszyński, K., Moores, M. T., and Stumpf-Fétizon, T. MCMC for multi-modal distributions. *arXiv preprint arXiv:2501.05908*, 2025.
- Li, X., Wu, K., Zhang, X., and Wang, H. B2opt: Learning to optimize black-box optimization with little budget. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 18502–18510, 2025.
- Li, Y., Guo, J., Wang, R., and Yan, J. T2t: From distribution learning in training to gradient search in testing for combinatorial optimization. *Advances in Neural Information Processing Systems*, 36:50020–50040, 2023.
- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=PqvMRDCJT9t>.
- Liu, D. C. and Nocedal, J. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1):503–528, 1989.
- Liu, Q. and Wang, D. Stein variational gradient descent: A general purpose bayesian inference algorithm. *Advances in neural information processing systems*, 29, 2016.
- Liu, X., Gong, C., and qiang liu. Flow straight and fast: Learning to generate and transfer data with rectified

- flow. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=XVjTT1nw5z>.
- Ma, Z., Guo, H., Gong, Y.-J., Zhang, J., and Tan, K. C. Toward automated algorithm design: A survey and practical guide to meta-black-box-optimization. *IEEE Transactions on Evolutionary Computation*, 2025.
- Martin, A., Manchester, I. R., and Furieri, L. Learning to optimize with guarantees: a complete characterization of linearly convergent algorithms. *arXiv preprint arXiv:2508.00775*, 2025.
- Mazé, F. and Ahmed, F. Diffusion models beat gans on topology optimization. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pp. 9108–9116, 2023.
- Molga, M. and Smutnicki, C. Test functions for optimization needs. *Test functions for optimization needs*, 101(48):32, 2005.
- Monga, V., Li, Y., and Eldar, Y. C. Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing. *IEEE Signal Processing Magazine*, 38(2): 18–44, 2021.
- Naser, M., Al-Bashiti, M. K., Tapeh, A. T. G., Naser, A., Kodur, V., Hawileh, R., Abdalla, J., Khodadadi, N., Gandomi, A. H., and Eslamlou, A. D. A review of benchmark and test functions for global optimization algorithms and metaheuristics. *Wiley Interdisciplinary Reviews: Computational Statistics*, 17(2):e70028, 2025.
- Neri, F. and Cotta, C. Memetic algorithms and memetic computing optimization: A literature review. *Swarm and Evolutionary Computation*, 2:1–14, 2012.
- Nguyen, H. and Donti, P. Fsnets: Feasibility-seeking neural network for constrained optimization with guarantees. *Advances in Neural Information Processing Systems*, 38: 39670–39708, 2026.
- Ollivier, Y., Arnold, L., Auger, A., and Hansen, N. Information-geometric optimization algorithms: A unifying picture via invariance principles. *Journal of Machine Learning Research*, 18(18):1–65, 2017.
- Oshin, A., Ghosh, R. V., Saravanas, A. D., and Theodorou, E. Deep flexQP: Accelerated nonlinear programming via deep unfolding. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=HL3TvE4Afm>.
- Park, S. and Van Hentenryck, P. Self-supervised primal-dual learning for constrained optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 4052–4060, 2023.
- Richter, L. and Berner, J. Improved sampling via learned diffusions. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=h4pNR0s006>.
- Robbins, H. and Monro, S. A stochastic approximation method. *The annals of mathematical statistics*, pp. 400–407, 1951.
- Sambharya, R. and Stellato, B. Learning algorithm hyperparameters for fast parametric convex optimization. *arXiv preprint arXiv:2411.15717*, 2024.
- Sambharya, R. and Stellato, B. Data-driven performance guarantees for classical and learned optimizers. *Journal of Machine Learning Research*, 26(171):1–49, 2025.
- Sambharya, R., Bok, J., Matni, N., and Pappas, G. Learning acceleration algorithms for fast parametric convex optimization with certified robustness. *arXiv preprint arXiv:2507.16264*, 2025.
- Samuel, N., Diskin, T., and Wiesel, A. Learning to detect. *IEEE Transactions on Signal Processing*, 67(10):2554–2564, 2019.
- Saravanas, A. D., Aoyama, Y., Zhu, H., and Theodorou, E. A. Distributed differential dynamic programming architectures for large-scale multiagent control. *IEEE Transactions on Robotics*, 39(6):4387–4407, 2023. doi: 10.1109/TRO.2023.3319894.
- Saravanas, A. D., Kuperman, H., Oshin, A., Abdul, A. T., Pacelli, V., and Theodorou, E. Deep distributed optimization for large-scale quadratic programming. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=hzuumhfYS0>.
- Shi, Y., Eberhart, R., et al. A modified particle swarm optimizer. In *Evolutionary computation proceedings*, volume 890, pp. 69–73, 1998.
- Shlezinger, N., Fu, R., and Eldar, Y. C. Deepsoft: Deep soft interference cancellation for multiuser mimo detection. *IEEE Transactions on Wireless Communications*, 20(2): 1349–1362, 2020.
- Shlezinger, N., Eldar, Y. C., and Boyd, S. P. Model-based deep learning: On the intersection of deep learning and optimization. *IEEE Access*, 10:115384–115398, 2022.
- Sinha, N., Chakrabarti, R., and Chattopadhyay, P. K. Evolutionary programming techniques for economic load dispatch. *IEEE Transactions on evolutionary computation*, 7(1):83–94, 2003.

- So, O., Karrer, B., Fan, C., Chen, R. T., and Liu, G.-H. Discrete adjoint matching. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Song, Y. and Ermon, S. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=PXTIG12RRHS>.
- Sucker, M., Fadili, J., and Ochs, P. Learning-to-optimize with pac-bayesian guarantees: Theoretical considerations and practical implementation. *Journal of Machine Learning Research*, 26(211):1–53, 2025.
- Sun, Z. and Yang, Y. Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in neural information processing systems*, 36:3706–3731, 2023.
- Theodoropoulos, P., Komianos, N., Pacelli, V., Liu, G.-H., and Theodorou, E. Feedback schrödinger bridge matching. In *International Conference on Learning Representations*, volume 2025, pp. 58939–58967, 2025.
- Theodoropoulos, P., Saravanos, A. D., Theodorou, E., and Liu, G.-H. Momentum multi-marginal schrödinger bridge matching. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=C7BIQRM57T>.
- Uehara, M., Zhao, Y., Hajiramezanali, E., Scalia, G., Eraslan, G., Lal, A., Levine, S., and Biancalani, T. Bridging model-based optimization and generative modeling via conservative fine-tuning of diffusion models. *Advances in Neural Information Processing Systems*, 37: 127511–127535, 2024.
- Van Hentenryck, P. Optimization learning. *arXiv preprint arXiv:2501.03443*, 2025.
- Varadhan, S. S. *Large deviations and applications*. SIAM, 1984.
- Vargas, F., Grathwohl, W. S., and Doucet, A. Denoising diffusion samplers. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=8pvnfTabu1f>.
- Wang, Z., So, O., Lee, K., and Theodorou, E. A. Adaptive risk sensitive model predictive control with stochastic search. In *Learning for Dynamics and Control*, pp. 510–522. PMLR, 2021.
- Wang, Z., Saravanos, A. D., Almubarak, H., So, O., and Theodorou, E. A. Sampling-based optimization for multi-agent model predictive control. *arXiv preprint arXiv:2211.11878*, 2022.
- Wierstra, D., Schaul, T., Glasmachers, T., Sun, Y., Peters, J., and Schmidhuber, J. Natural evolution strategies. *The Journal of Machine Learning Research*, 15(1):949–980, 2014.
- Williams, G., Aldrich, A., and Theodorou, E. A. Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics*, 40(2):344–357, 2017.
- Wright, S., Nocedal, J., et al. Numerical optimization. *Springer Science*, 35(67-68):7, 1999.
- Xie, Y. and Cheng, X. Flow-based generative models as iterative algorithms in probability space: An intuitive mathematical framework [special issue on the mathematics of deep learning]. *IEEE Signal Processing Magazine*, 43(2):37–50, 2026.
- Zambrano-Bigiarini, M., Clerc, M., and Rojas, R. Standard particle swarm optimisation 2011 at cec-2013: A baseline for future pso improvements. In *2013 IEEE congress on evolutionary computation*, pp. 2337–2344. IEEE, 2013.
- Zhang, K., Gool, L. V., and Timofte, R. Deep unfolding network for image super-resolution. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 3217–3226, 2020.
- Zhang, Q. and Chen, Y. Path integral sampler: A stochastic control approach for sampling. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=_uCb2ynRu7Y.
- Zhigljavsky, A. *Stochastic Global Optimization*, pp. 2672–2675. Springer Berlin Heidelberg, Berlin, Heidelberg, 2025. ISBN 978-3-662-69359-9. doi: 10.1007/978-3-662-69359-9_654. URL https://doi.org/10.1007/978-3-662-69359-9_654.
- Zhou, E. and Hu, J. Gradient-based adaptive stochastic search for non-differentiable optimization. *IEEE Transactions on Automatic Control*, 59(7):1818–1832, 2014.

A. Extended Related Work

Sampling-Based Optimization. Sampling-based optimization algorithms rely on viewing optimization problems as the task of constructing and refining a proposal distribution to sample from low-cost regions of the objective landscape. This perspective has given rise to various algorithms such as the Cross-Entropy Method (CEM) (De Boer et al., 2005), which frames optimization as an iterative density estimation task. By minimizing the Kullback-Leibler (KL) divergence between a parameterized proposal typically Gaussian distribution and an implicit target distribution defined over a top-performing fraction of “elite” samples, CEM sequentially concentrates its probability mass around lower-cost regions. Nevertheless, CEM-type algorithms are known to suffer from premature mode collapse and convergence issues which limits its applicability, especially in high-dimensional non-convex settings. Similar algorithms such as stochastic search (Zhou & Hu, 2014; Wang et al., 2021), the model predictive path integral algorithm (Williams et al., 2017; Wang et al., 2022), etc. can also be found in various optimization, machine learning and engineering areas.

A closely related family of methods are evolutionary strategies (ES), which similarly maintain and adapt a search distribution—typically Gaussian—but refine its parameters through stochastic, gradient-free perturbations weighted by sampled fitness. One of the most popular methods is the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) (Hansen & Ostermeier, 2001; Hansen, 2016), a state-of-the-art black-box optimizer that adapts the full covariance of the proposal to capture local curvature and variable correlations, rendering it particularly effective on ill-conditioned, non-separable landscapes. More recent formulations such as Natural Evolution Strategies (NES) (Wierstra et al., 2014) recast these updates as natural-gradient ascent on the expected fitness, offering a principled information-geometric foundation (Ollivier et al., 2017) that unifies many ES and estimation-of-distribution algorithms. Despite their robustness, these approaches rely on hand-crafted, myopic update rules and a restrictive Gaussian parameterization, motivating learning-based proposals capable of representing richer, multimodal distributions and algorithmic updates.

Particle Swarm Optimization. Particle swarm optimization (PSO) is a population-based, derivative-free paradigm in which candidate solutions explore the search space via velocities that blend each particle’s own best location with the swarm’s best (Kennedy & Eberhart, 1995). Such approaches have been later refined with inertia weights (Shi et al., 1998), constriction coefficients (Clerc & Kennedy, 2002), and the Standard PSO 2011 variant (Zambrano-Bigiarini et al., 2013). Its update rule, however, is a hand-designed linear combination of cognitive and social terms with hand-tuned coefficients. FlowOptimizer can be seen as a learned generalization: it likewise evolves a population through a velocity field, but replaces PSO’s fixed heuristic with a learned velocity field, adapting the population’s trajectories to the problem family rather than relying on static swarm dynamics.

Gradient-Based Non-Convex Optimization. Gradient-based optimization methods rely fundamentally on local derivative information to guide their updates (Wright et al., 1999). Although their theoretical foundations and convergence guarantees are rooted in the convex optimization setting (Boyd & Vandenberghe, 2004), gradient-based methods are very commonly used for navigating the complex, high-dimensional non-convex loss surfaces. In practice, the workhorse of large-scale learning is stochastic gradient descent (SGD) (Robbins & Monro, 1951; Bottou et al., 2018), which substitutes the full gradient with cheap, noisy minibatch estimates. To improve conditioning and alleviate the burden of manual step-size tuning, a family of adaptive methods rescales updates using accumulated gradient statistics, including AdaGrad (Duchi et al., 2011), which adapts per-coordinate learning rates from the running sum of squared gradients, and Adam (Kingma & Ba, 2015), which couples momentum with an exponentially decaying second-moment estimate and has become the de facto optimizer for training deep networks. Beyond such first-order schemes, second-order methods incorporate curvature information for faster local convergence: Newton’s method rescales the gradient by the inverse Hessian (Wright et al., 1999), quasi-Newton schemes such as BFGS and its limited-memory variant L-BFGS (Liu & Nocedal, 1989) build low-rank Hessian approximations from successive gradients, and structured variants such as Differential Dynamic Programming (Jacobson & Mayne, 1970) exploit problem-specific structure (e.g., the temporal structure of dynamics in trajectory optimization) have been shown to be quite effective in large-scale non-convex optimization settings (Saravanos et al., 2023). Yet all such methods remain fundamentally local, relying on derivative information that offers little guidance on the rugged, high-dimensional non-convex landscapes encountered in practice.

Learning-to-Optimize and Deep Unfolding. Learning-to-optimize methodologies aim to learn optimization algorithms by training on problem data (Chen et al., 2022; Shlezinger et al., 2022; Amos et al., 2023). A first class of approaches learns *optimization proxies* that map problem instances directly to near-optimal solutions (Amos et al., 2023); these are especially effective for parametric problems that must be solved repeatedly, where supervised and self-supervised primal-dual schemes

recover high-quality feasible solutions orders of magnitude faster than classical solvers (Fioretto et al., 2020; Park & Van Hentenryck, 2023; Van Hentenryck, 2025). A second, more structured class is *deep unfolding*, which unrolls iterative optimization algorithms as sequential layers of a neural network, enabling end-to-end learning of algorithmic parameters (Monga et al., 2021; Zhang et al., 2020). Recent works further advance this paradigm through more expressive architectures and learning strategies (Sambharya & Stellato, 2024; Saravanos et al., 2025; Oshin et al., 2026), improving performance and generalization across diverse tasks. A third, complementary line treats optimization itself as a *differentiable* component, embedding parametric programs as implicit layers and enforcing hard constraints through differentiable feasibility steps (Amos & Kolter, 2017; Donti et al., 2017; 2021; Nguyen & Donti, 2026).

Generative Modeling. Generative models learn to transport a simple reference distribution to a complex data distribution. *Diffusion models* cast this as the time reversal of a noising process, training a network to denoise or estimate the score of the noised marginals (Ho et al., 2020; Song & Ermon, 2019), with a continuous-time SDE formulation and an equivalent probability-flow ODE (Song et al., 2021). *Flow-based* models instead learn a velocity field whose ODE carries the reference to the target (Chen et al., 2018), a view closely tied to optimal transport and Wasserstein gradient flows (Xie & Cheng, 2026). A growing body of work repurposes such generative models as optimization processes, recasting the search for high-quality solutions as sampling from a target distribution concentrated on low-cost regions of the objective. Their appeal lies in representing rich, multimodal solution families far beyond the unimodal Gaussians of classical sampling-based methods, though their use in end-to-end optimization is complicated by the implicit nature of the target and the lack of ground-truth solutions. Recent efforts couple generative sampling with feasibility or control structure (Idoko & Singh, 2025; Kurtz & Burdick, 2025), frame optimization or constrained transport as reward-tilted, KL-regularized, or marginal-constrained sampling solved via stochastic optimal control (Domingo-Enrich et al., 2025; So et al., 2026; Theodoropoulos et al., 2025; 2026), and more broadly seek to bridge generative modeling and optimization across discrete and continuous domains (Uehara et al., 2024; Kong et al., 2025; Mazé & Ahmed, 2023; Xie & Cheng, 2026). The field nonetheless remains nascent, with most approaches still facing scalability and expressiveness limitations.

Meta-Black-Box Optimization. A parallel line of L2O research is Meta-Black-Box Optimization (MetaBBO), which uses meta-learning to automatically design or configure optimizers that generalize across a distribution of problems (Ma et al., 2025). MetaBBO typically adopts a bi-level structure: a meta-level policy dictates the algorithm selection, hyperparameter configuration, update rules, or solution manipulation of a low-level black-box optimizer, often a population-based evolutionary algorithm (Ma et al., 2025; Chen et al., 2024; Li et al., 2025). The defining assumption of this setting is that objectives are *non-differentiable* and expensive to query, so performance is measured under a function-evaluation budget and the low-level optimizer is derivative-free. FlowOptimizer occupies the complementary regime: it assumes access to differentiable objectives and targets low wall-clock cost rather than evaluation count, learning a velocity field by flow matching and refining it through deep unfolding.

B. Experiment Details

B.1. General Information

For all algorithms, we use a population size $P = 20$. Unless otherwise specified, in the main experiments, the previous population was directly fed into the velocity field model. All experiments were performed on an RTX 4090 GPU 24GB.

B.2. Architecture Details

In all experiments, the velocity field is parameterized by a permutation-equivariant neural network with hidden dimension $h = 128$. Each population member $z_t^{(i)} \in \mathbb{R}^n$ is first embedded using a linear layer $\mathbb{R}^n \rightarrow \mathbb{R}^{128}$. The per-member context features are concatenated and embedded using a separate linear layer to $\mathbb{R}^{128}$. The state and context embeddings are concatenated to form a 256-dimensional representation per population member. Population interactions are modeled using a single multi-head self-attention layer with 4 heads and embedding dimension 256, followed by a residual connection and layer normalization. The resulting per-member features are concatenated with global conditioning features shared across the population, including the scalar flow time, optional averaged gradient information, and summary statistics from a history buffer of length $H = 3$. History features consist of the mean and standard deviation of past populations. The final per-member velocity is produced by an MLP with 3 hidden layers of width 128, each followed by layer normalization and ELU activations, and a final linear layer mapping to $\mathbb{R}^n$. In the dimension-agnostic variant, in order to capture cross-dimensional

coupling without introducing n -dependent parameters, we add a low-rank mixing term

$$v_{j,i} = v_{j,i}^{\text{loc}} + \langle a_{j,i}, b_j \rangle, \quad a_{j,i}, b_j \in \mathbb{R}^r,$$

where $r \ll n$ and both $a_{j,i}$ and b_j are produced by shared networks from $(z_{t,j,i}, h_j, t)$ (and gradients when available).

B.3. Training Details

For all problem classes, we use 1000 problems for training, 200 for validation and 200 held-out ones for testing. The batch size is set to 50. The Adam optimizer with learning rate 10^{-4} is used in all experiments.

Pre-training via flow matching loss. In the pre-training phase, we construct the flow matching dataset pairs as follows. We sample a base population of P points via a Gaussian distribution with same mean and covariance as the initializations μ^0 and Σ^0 for each problem class. Then, the improvement procedure takes place by running CMA-ES and selecting pairs across the trajectory with pair offset $\Delta = 10$.

Objective-driven end-to-end fine-tuning. During the fine-tuning phase, the balancing parameter in the training loss is set to $\alpha = 0.8$. Initially, we follow a progressive training scheme where each iteration-specific flow model is trained using the sum of per-iteration losses up to that iteration, while the others are frozen. At each cycle each iteration-specific model is trained for 10 epochs except for the last one which is trained for 60 epochs. This process takes place for 1800 epochs. During the last 200 epochs all models are trained simultaneously using the total loss across iterations.

B.4. Baselines

Random Search (RS). Random Search (RS) samples candidate solutions from a fixed distribution and returns the one with the lowest objective value. Across all problems, RS samples from a uniform distribution supported on the entire domain of the function. RS serves as a fundamental naive baseline, included to verify that the gains of more sophisticated methods stem from exploiting problem structure rather than from sample coverage alone.

Cross-Entropy Method (CEM). The Cross-Entropy Method (CEM) is a sampling-based optimization algorithm that iteratively updates a parametric sampling distribution to concentrate probability mass on regions of the search space that correspond to low objective values. At iteration k , a population of P samples $\{x_j^k\}_{j=1}^P$ is drawn from a Gaussian distribution $x_j^k \sim \mathcal{N}(\mu^k, \Sigma^k)$, and then each sample is evaluated to obtain the values $f(x_j^k)$. Then, the next step is to update the distribution parameters μ^{k+1}, Σ^{k+1} to be used in the next iteration. In the most common ‘‘hard’’ variant of CEM, an elite set $\mathcal{E}^k \subset \{1, \dots, P\}$ consisting of the P_e samples with the lowest objective values is selected. The distribution parameters are then updated by maximum likelihood estimation over the elite samples as follows

$$\mu^{k+1} = \frac{1}{P_e} \sum_{j \in \mathcal{E}^k} x_j^k, \quad \Sigma^{k+1} = \frac{1}{P_e} \sum_{j \in \mathcal{E}^k} (x_j^k - \mu^{k+1})(x_j^k - \mu^{k+1})^\top. \quad (14)$$

Across all experiments, the elite fraction is set to the reasonable default $P_e/P = 0.2$.

Covariance Matrix Adaptation Evolution Strategy (CMA-ES). CMA-ES is a population-based evolutionary strategy that adapts a Gaussian search distribution by updating its mean, covariance matrix and global step size. At iteration k , a population of P samples is generated as

$$x_j^k = \mu^k + \sigma^k y_j^k, \quad y_j^k \sim \mathcal{N}(0, \Sigma^k), \quad j = 1, \dots, P, \quad (15)$$

where $\mu^k \in \mathbb{R}^n$ is the mean, $\Sigma^k \in \mathbb{R}^{n \times n}$ is the covariance matrix, and $\sigma^k > 0$ is a global step-size parameter.

The samples are evaluated and ranked according to their objective values. Let $\{x_{(j)}^k\}_{j=1}^{P_e}$ denote the P_e best-performing samples. The mean is updated via weighted recombination:

$$\mu^{k+1} = \sum_{j=1}^{P_e} w_j x_{(j)}^k, \quad \sum_{j=1}^{P_e} w_j = 1, \quad w_j > 0. \quad (16)$$

The covariance matrix is updated to reflect the directions of successful search steps:

$$\Sigma^{k+1} = (1 - c_\mu)\Sigma^k + c_\mu \sum_{j=1}^{P_e} w_j \frac{(x_{(j)}^k - \mu^k)(x_{(j)}^k - \mu^k)^\top}{(\sigma^k)^2}, \quad (17)$$

where $c_\mu \in (0, 1)$ is a learning rate. The global step size σ^k is adapted using cumulative step-size adaptation, which increases σ^k when consecutive steps are aligned and decreases it otherwise.

Particle Swarm Optimization (PSO). Particle Swarm Optimization (PSO) is a population-based metaheuristic in which a swarm of P particles explores the search space by combining each particle's own experience with that of the swarm. We use the standard SPSO-2011 configuration (Zambrano-Bigiarini et al., 2013). As with the other algorithms, the initial particle positions $\{x_j^0\}_{j=1}^P$ are drawn from a Gaussian $\mathcal{N}(\mu, \Sigma)$ matching the initialization used by the other algorithms for the same problem class, and the velocities $\{v_j^0\}_{j=1}^P$ are initialized to zero. Each particle j maintains its personal best position p_j^k , i.e. the lowest-objective point it has visited up to iteration k , and the swarm maintains the global best position g^k over all particles. At iteration k , the velocity and position of each particle are updated as

$$v_j^{k+1} = w v_j^k + c_1 r_1 \odot (p_j^k - x_j^k) + c_2 r_2 \odot (g^k - x_j^k), \quad (18)$$

$$x_j^{k+1} = x_j^k + v_j^{k+1}, \quad j = 1, \dots, P, \quad (19)$$

where $\odot$ denotes elementwise multiplication, $r_1, r_2 \sim \mathcal{U}([0, 1]^n)$ are sampled independently at every step, w is the inertia weight, and c_1, c_2 are the cognitive and social coefficients respectively. We adopt the SPSO-2011 defaults $w = 1/(2 \ln 2) \approx 0.7213$ and $c_1 = c_2 = 1/2 + \ln 2 \approx 1.1931$, which are not tuned. The algorithm returns the global best position found across all particles and iterations.

Multi-Start Gradient Descent (MS-GD). Multi-Start Gradient Descent (MS-GD) is a simple strategy that mitigates sensitivity to initialization in gradient-based optimization by running multiple independent gradient descent trajectories from different starting points. Specifically, we draw P initial points $\{x_j^0\}_{j=1}^P$ from a Gaussian distribution $\mathcal{N}(\mu, \Sigma)$, matching the initialization used by the other algorithms for the same problem class. Each initialization is then evolved independently using gradient descent updates of the form

$$x_j^{k+1} = x_j^k - \eta \nabla f(x_j^k), \quad j = 1, \dots, P, \quad (20)$$

where $\eta > 0$ is a fixed step size. The algorithm returns the best solution across all trajectories. For tuning, we perform a grid search over $\eta \in \{10^{-3}, 2 \cdot 10^{-3}, 5 \cdot 10^{-3}, \dots, 1.0, 2.0, 5.0\}$.

Multi-Start Nesterov Accelerated Gradient Descent (MS-NAG). Multi-Start Nesterov Accelerated Gradient Descent (MSNAG) extends MS-GD by incorporating acceleration into each trajectory. As in MS-GD, we draw P initial points $\{x_j^0\}_{j=1}^P$ from a Gaussian distribution $\mathcal{N}(\mu, \Sigma)$, matching the initialization used by the other algorithms for the same problem class. Each trajectory is then evolved independently according to the Nesterov accelerated updates

$$y_j^k = x_j^k + \beta (x_j^k - x_j^{k-1}), \quad (21)$$

$$x_j^{k+1} = y_j^k - \eta \nabla f(y_j^k), \quad j = 1, \dots, P, \quad (22)$$

where $\eta > 0$ is a fixed step size and $\beta \in [0, 1)$ is the momentum coefficient. The algorithm returns the best solution across all trajectories. We set the momentum coefficient $\beta = 0.9$ and tune the step size η through performing a grid search over $\eta \in \{10^{-3}, 2 \cdot 10^{-3}, 5 \cdot 10^{-3}, \dots, 1.0, 2.0, 5.0\}$.

Multi-Start Adam (MS-Adam). Multi-Start Adam (MS-Adam) extends MS-GD by replacing the plain gradient update with the Adam adaptive moment estimation scheme. As before, we draw P initial points $\{x_j^0\}_{j=1}^P$ from a Gaussian distribution $\mathcal{N}(\mu, \Sigma)$, matching the initialization used by the other algorithms for the same problem class. Each trajectory maintains first- and second-moment estimates m_j^k, v_j^k (initialized at zero) and is evolved independently according to the

Adam updates

$$g_j^k = \nabla f(x_j^k), \quad (23)$$

$$m_j^{k+1} = \beta_1 m_j^k + (1 - \beta_1) g_j^k, \quad (24)$$

$$v_j^{k+1} = \beta_2 v_j^k + (1 - \beta_2) (g_j^k)^2, \quad (25)$$

$$\hat{m}_j^{k+1} = \frac{m_j^{k+1}}{1 - \beta_1^{k+1}}, \quad \hat{v}_j^{k+1} = \frac{v_j^{k+1}}{1 - \beta_2^{k+1}}, \quad (26)$$

$$x_j^{k+1} = x_j^k - \eta \frac{\hat{m}_j^{k+1}}{\sqrt{\hat{v}_j^{k+1} + \epsilon}}, \quad j = 1, \dots, P, \quad (27)$$

where $\eta > 0$ is the learning rate, all operations are applied elementwise, and $\epsilon > 0$ is a small constant for numerical stability (fixed to 10^{-8}). The moment decay rates are fixed to their standard values $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The algorithm returns the best solution across all trajectories. The learning rate η is tuned through performing a grid search over $\eta \in \{10^{-3}, 2 \cdot 10^{-3}, 5 \cdot 10^{-3}, \dots, 1.0, 2.0, 5.0\}$.

Multi-Start Newton (MS-Newton). Multi-Start Newton (MS-Newton) extends MS-GD by incorporating second-order curvature information into each trajectory. As before, we draw P initial points $\{x_j^0\}_{j=1}^P$ from a Gaussian distribution $\mathcal{N}(\mu, \Sigma)$, matching the initialization used by the other algorithms for the same problem class. Each trajectory is then evolved independently according to the damped Newton updates

$$x_j^{k+1} = x_j^k - (\nabla^2 f(x_j^k) + \lambda I)^{-1} \nabla f(x_j^k), \quad j = 1, \dots, P, \quad (28)$$

where $\lambda \geq 0$ is a damping (Levenberg–Marquardt) parameter that regularizes the Hessian, ensuring the linear system is well-conditioned and the step remains a descent direction when $\nabla^2 f(x_j^k)$ is indefinite, as is common for the nonconvex problem classes considered here. The algorithm returns the best solution across all trajectories. The damping parameter is set to a reasonable default $\lambda = 10^{-3}$.

Multi-Start Variant of Learning-to-Optimize Gradient Descent (L2O-GD). We also consider a L2O variant of MSGD where we learn the “optimal” stepsizes for every iteration of the gradient descent updates as in (Sambharya & Stellato, 2024). During inference, we draw P samples from the same initial distributions as the rest of the algorithms, and apply the same learned updates on all samples. At the end, the algorithm returns the best solution across all trajectories.

B.5. Standard Optimization Benchmarks

Here, we provide a detailed description of the non-convex optimization benchmark functions used in our experiments, along with their configurations. For comprehensive references on such functions, the reader is referred to (Molga & Smutnicki, 2005; Naser et al., 2025). A 2D visualization of the functions used (Ackley, Rastrigin and Levy) is provided in Fig. 6.

Ackley function. The Ackley function is given by

$$f(x) = -a \exp\left(-b \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - c_i)^2}\right) - \exp\left(\frac{1}{n} \sum_{i=1}^n \cos(\omega(x_i - c_i))\right) + a + \exp(1), \quad (29)$$

where the parameter $a > 0$ controls the amplitude of the outer exponential, $b > 0$ governs the decay rate of the first exponential term, $\omega > 0$ is the cosine frequency, controlling the density of local minima and $c = [c_1, \dots, c_n] \in \mathbb{R}^n$ is a shift vector. The ground-truth global optimum of this function is found at $x^* = c$ with $f^* = 0$.

Across our experiments, we consider the domain to be $x \in [-30, 30]^n$, and the problem parameters to be randomly drawn from $a \sim \text{LogUniform}[10, 40]$, $b \sim \text{LogUniform}[0.1, 0.4]$, $\omega \sim \text{Uniform}[\pi, 2\pi]$ and $c \sim \text{Uniform}[-3, 3]^n$. For all algorithms, the particles of the initial population are randomly drawn from $x_0^j \sim \mathcal{N}(\mu_0, \Sigma_0)$, $j = 1, \dots, P$, where $\mu_0 \sim \text{Uniform}[-25, 25]^n$ and $\Sigma_0 = I_n$.

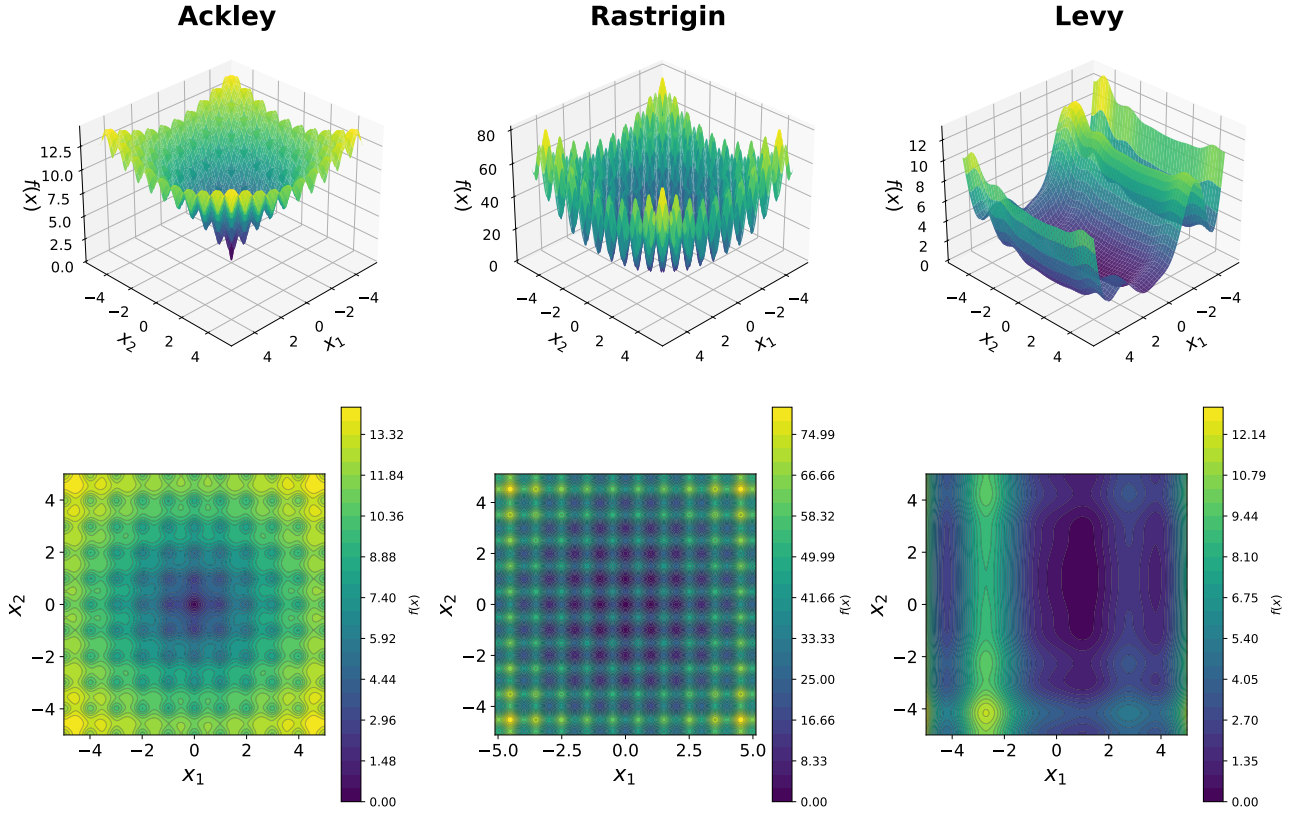


Figure 6. **2D Illustration of Ackley, Rastrigin and Levy Functions.** These optimization benchmarks are highly non-convex, with their complexity further growing as dimension increases.

Rastrigin function. The Rastrigin function is given by

$$f(x) = an + \sum_{i=1}^n ((x_i - c_i)^2 - a \cos(2\pi(x_i - c_i))), \quad (30)$$

where the parameter $a > 0$ controls the amplitude of the cosine term and $c = [c_1, \dots, c_n] \in \mathbb{R}^n$ is a shift vector. The ground-truth global optimum is found at $x^* = c$ with $f^* = 0$.

Across our experiments, we consider the domain to be $x \in [-10, 10]^n$, and the problem parameters are randomly drawn from $a \sim \text{Uniform}[0, 5]$ and $c \sim \text{Uniform}[-3, 3]^n$. For all algorithms, the particles of the initial population are randomly drawn from $x_0^j \sim \mathcal{N}(\mu_0, \Sigma_0)$, $j = 1, \dots, P$, where $\mu_0 \sim \text{Uniform}[-8, 8]^n$ and $\Sigma_0 = I_n$.

Levy function. The Levy function is given by

$$f(x) = \sin^2(\pi w_1) + \sum_{i=1}^{n-1} (w_i - 1)^2 \left(1 + 10 \sin^2(\pi w_i + 1) \right) + (w_n - 1)^2 \left(1 + \sin^2(2\pi w_n) \right), \quad (31)$$

where

$$w_i = 1 + \frac{(x_i - c_i)/\alpha - 1}{4}, \quad i = 1, \dots, n, \quad (32)$$

where the parameter $\alpha > 0$ is a scaling factor that stretches or compresses the input space, and $c = [c_1, \dots, c_n] \in \mathbb{R}^n$ is a shift vector. The ground-truth global optimum is located at $x^* = c$ with $f^* = 0$.

Across our experiments, we consider the domain $x \in [-10, 10]^n$, and the problem parameters are randomly drawn from $\alpha \sim \text{LogUniform}[0.2, 5]$ and $c \sim \text{Uniform}[-3, 3]^n$. For all algorithms, the particles of the initial population are randomly drawn from $x_0^j \sim \mathcal{N}(\mu_0, \Sigma_0)$, $j = 1, \dots, P$, where $\mu_0 \sim \text{Uniform}[-8, 8]^n$ and $\Sigma_0 = I_n$.

B.6. Real-World Application Problems

Robotic Arm Inverse Kinematics. We also consider an inverse kinematics task involving a planar robot arm with D degrees of freedom. The decision variable is the vector of joint angles $\Theta \in [-\pi, \pi]^D$, and the objective is to find a configuration such that the end-effector position $p_{ee}(\Theta)$ coincides with a target location p_{target} while avoiding collisions with a set of M circular obstacles. The full objective is defined as a weighted sum of a target-reaching term and a collision-avoidance penalty,

$$f(\Theta) = \|p_{ee}(\Theta) - p_{\text{target}}\|_2^2 + \lambda \sum_{i=1}^N \sum_{j=1}^M \text{ReLU}(R_j - \|s_i(\Theta) - o_j\|_2)^2, \quad (33)$$

where $p_{ee}(\Theta)$ is obtained via the forward kinematics chain, and the second term penalizes penetrations into obstacles by discretizing the arm into N sample points $s_i(\Theta)$ located along the links. Each obstacle is modeled by a center o_j and a radius R_j , and the ReLU operator ensures that the penalty is active only when a sample point lies inside an obstacle. The scalar λ balances the target-reaching and collision-avoidance objectives. This optimization landscape is highly non-convex and multimodal, as feasible solutions often require navigating around obstacles, forcing the optimizer to escape local minima corresponding to configurations that are close to the target but infeasible due to collisions.

Power Grid Economic Load Dispatch. We consider the Economic Load Dispatch (ELD) problem in power systems, a classical non-convex optimization problem that aims to determine the power output of multiple generators so as to minimize total generation cost while satisfying demand constraints (Sinha et al., 2003). The objective function is given by

$$f(x) = \sum_{i=1}^{N_g} [a_i x_i^2 + b_i x_i + c_i + |d_i \sin(e_i(P_i^{\min} - x_i))|] + \lambda \left(\sum_{i=1}^{N_g} x_i - P_D \right)^2 \quad (34)$$

where N_g is the number of generators and $x = [x_1; \dots; x_{N_g}] \in \mathbb{R}^{N_g}$ is the vector of power outputs. Each generator has the fuel cost coefficients a_i, b_i, c_i , the valve-point loading coefficients d_i, e_i and a minimum generation limit $P_i^{\min}$. In addition, the generators need to meet the total power demand P_D .

Supply Chain Multi-Source Weber Problem. We consider the Multi-Source Weber Problem (MSWP), a classical facility location problem arising in supply chain and logistics applications (Brimberg et al., 2008). The objective is to determine the locations of K facilities so as to minimize the total weighted distance to a set of M customers, and is given by

$$f(X) = \sum_{j=1}^M w_j \min_{k=1 \dots K} \|x_k - a_j\|_2 \quad (35)$$

where $X = \{x_1, \dots, x_K\}$ are the coordinates of the K facilities (optimization variables), a_j are the fixed locations of the M customers, w_j is the demand weight of customer j (usually set to 1.0).

We consider the smoothed objective using the SoftMin operator as follows:

$$f(X) = \sum_{j=1}^M w_j \left(-\frac{1}{\gamma} \log \sum_{k=1}^K \exp(-\gamma \|x_k - a_j\|_2) \right) \quad (36)$$

where $\gamma > 0$ is a temperature parameter controlling the smoothness of the approximation. As $\gamma \rightarrow \infty$, the SoftMin function approaches the hard minimum, $f_\gamma(X) \rightarrow f(X)$, recovering the original non-smooth objective. This relaxation renders the landscape differentiable everywhere, enabling the application of gradient-based baselines such as Nesterov momentum and Adam, while preserving the non-convex structure inherent to the facility location problem.

C. Additional Experiments

C.1. 1D Flat Middle Region Example with Identical Initializations

In this example, we evaluate the ability of FlowOptimizer to navigate towards the optimum in cases where different problem instances share an identical initial population located in a flat region of the objective landscape. This is a quite interesting example, as it studies how our flow-based model effectively explores the landscape despite its deterministic nature. We consider the following function:

$$f(x) = \underbrace{\frac{1 - \text{sgn}(\mu) \text{sgn}(x)}{2}}_{\text{bad-side mask}} \left[\kappa x^2 + B, \varepsilon(x) \sin(\omega x) \right] - A \exp \left(-\frac{(x - \mu)^2}{2\sigma^2} \right) \quad (37)$$

where $\varepsilon(x) = 1 - (1 + (x/W)^8)^{-1}$, with fixed $\kappa = 0.05$, $B = 0.12$, $\omega = 2.8$, $W = 1.6$, and per-instance parameters drawn uniformly as $\mu \sim \mathcal{U}(2.5, 4.0)$ (sign randomized), $A \sim \mathcal{U}(1.2, 2.8)$, $\sigma \sim \mathcal{U}(0.25, 0.40)$. The domain is $[-5.5, 5.5]$ and populations are initialized with mean $\sim \mathcal{U}(-0.5, 0.5)$ and std 0.2, placing them squarely in the flat middle region.

We consider five distinct objective functions that share the same initialization (Fig. 7). As expected, the first iteration produces identical population updates across all instances, since both the population and the available historical information are identical. However, the learned update policy has acquired the ability to actively spread particles when operating in flat regions. Consequently, small differences in objective evaluations become amplified over subsequent iterations, leading to increasingly distinct population trajectories across problem instances. By iteration 4, the populations have already diverged substantially, allowing the optimizer to identify promising regions of the search space. As shown in Fig. 7, the trajectories subsequently converge toward the correct optima of their respective objective functions.

This experiment illustrates that identical initial populations do not imply identical optimization trajectories beyond the first iteration. The incorporation of historical context enables FlowOptimizer to progressively disambiguate problem instances and adapt its search behavior accordingly.

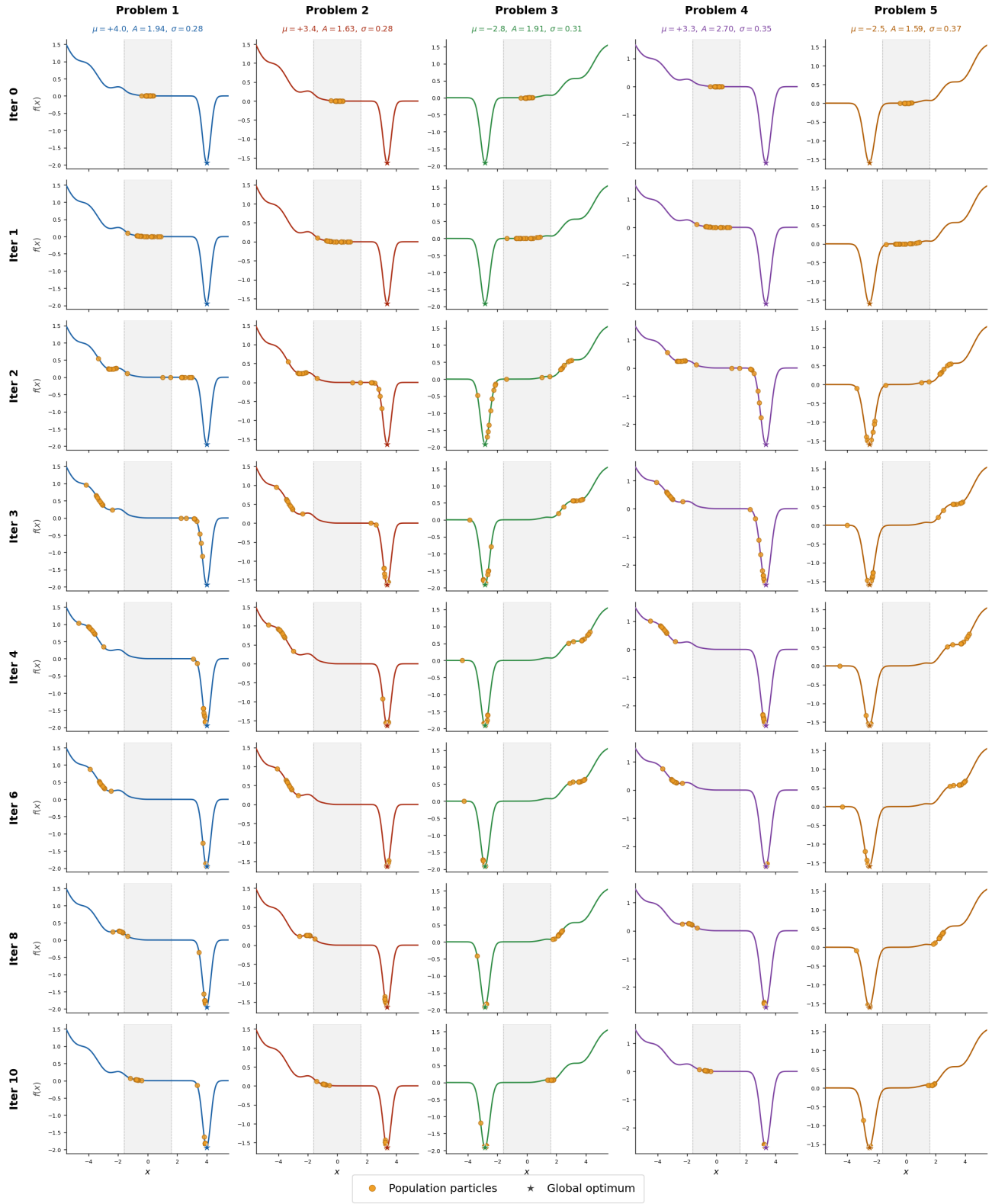


Figure 7. 1D Flat Middle Region Example with Identical Initial Populations.

C.2. Detailed 2D Ackley Example

Next, we demonstrate a 2D Ackley example to clearly illustrate the underlying velocity fields FlowOptimizer learns as well as the trajectories of the populations. In Figs. 8 and 9, the learned velocity fields and population iterates of our methodology are illustrated across different iterations. Notably, as the iterations progress, the velocity field tends to point towards the global optimum despite the very high number of local minima in the landscape.

In Fig. 10, we see the sample trajectories for all methods for a specific problem instance, while Figs. 11 and 12 show the samples of each method across iterations. This illustration highlights the ability of FlowOptimizer to drive the points towards the optimum fast in contrast with the considered baselines.

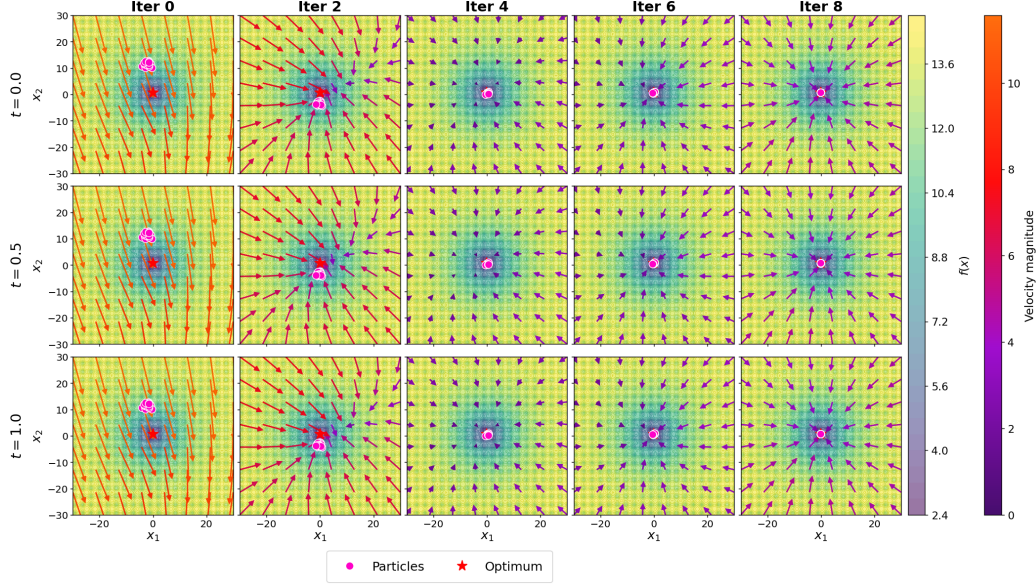


Figure 8. Visualization of FlowOptimizer Velocity Field - 2D Ackley Problem - Full Domain.

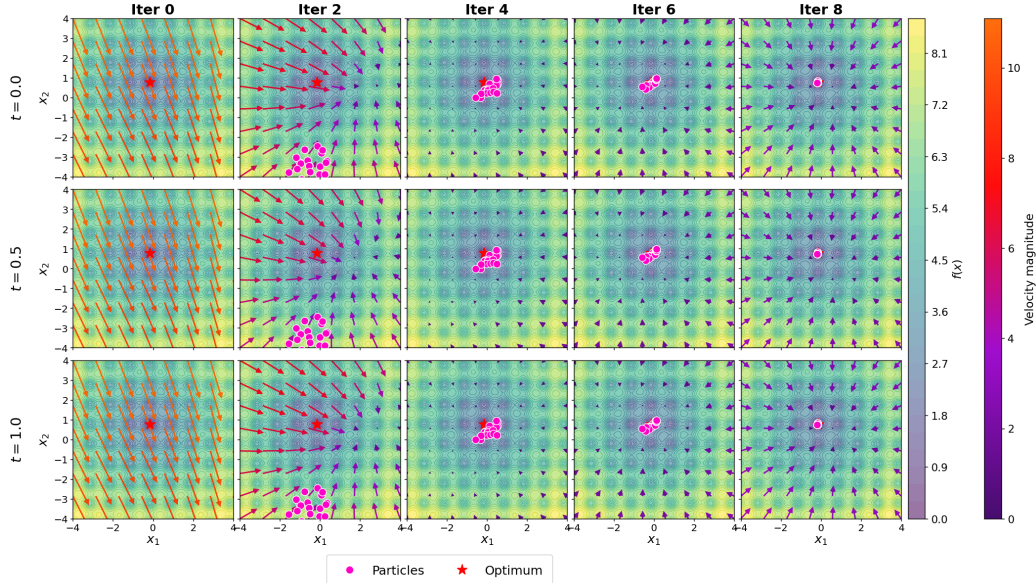


Figure 9. Visualization of FlowOptimizer Velocity Field - 2D Ackley Problem - Zoomed into $[-4, 4]^2$.

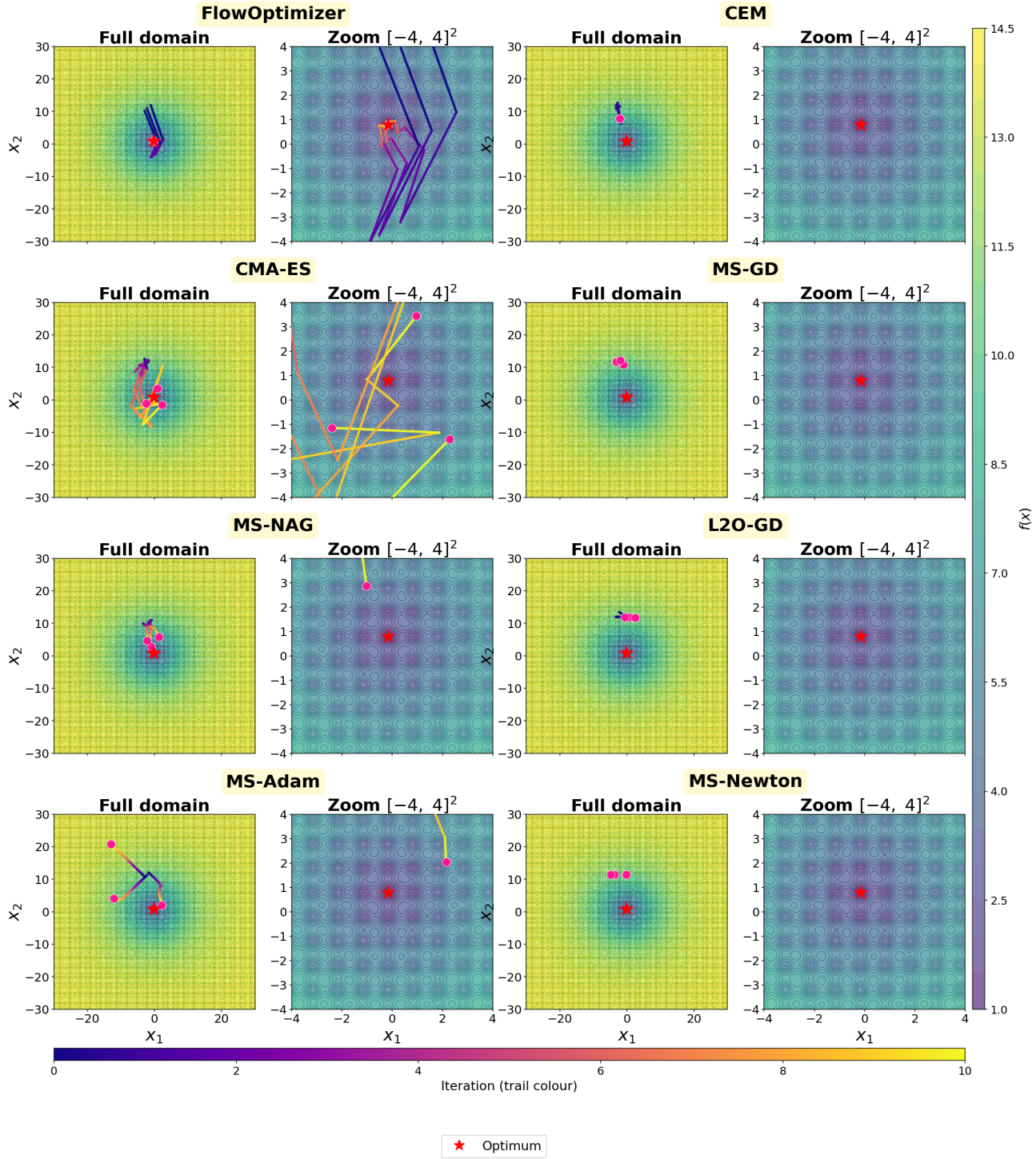


Figure 10. Visualization of Trajectories of Three Selected Particles Per Method - 2D Ackley Problem.

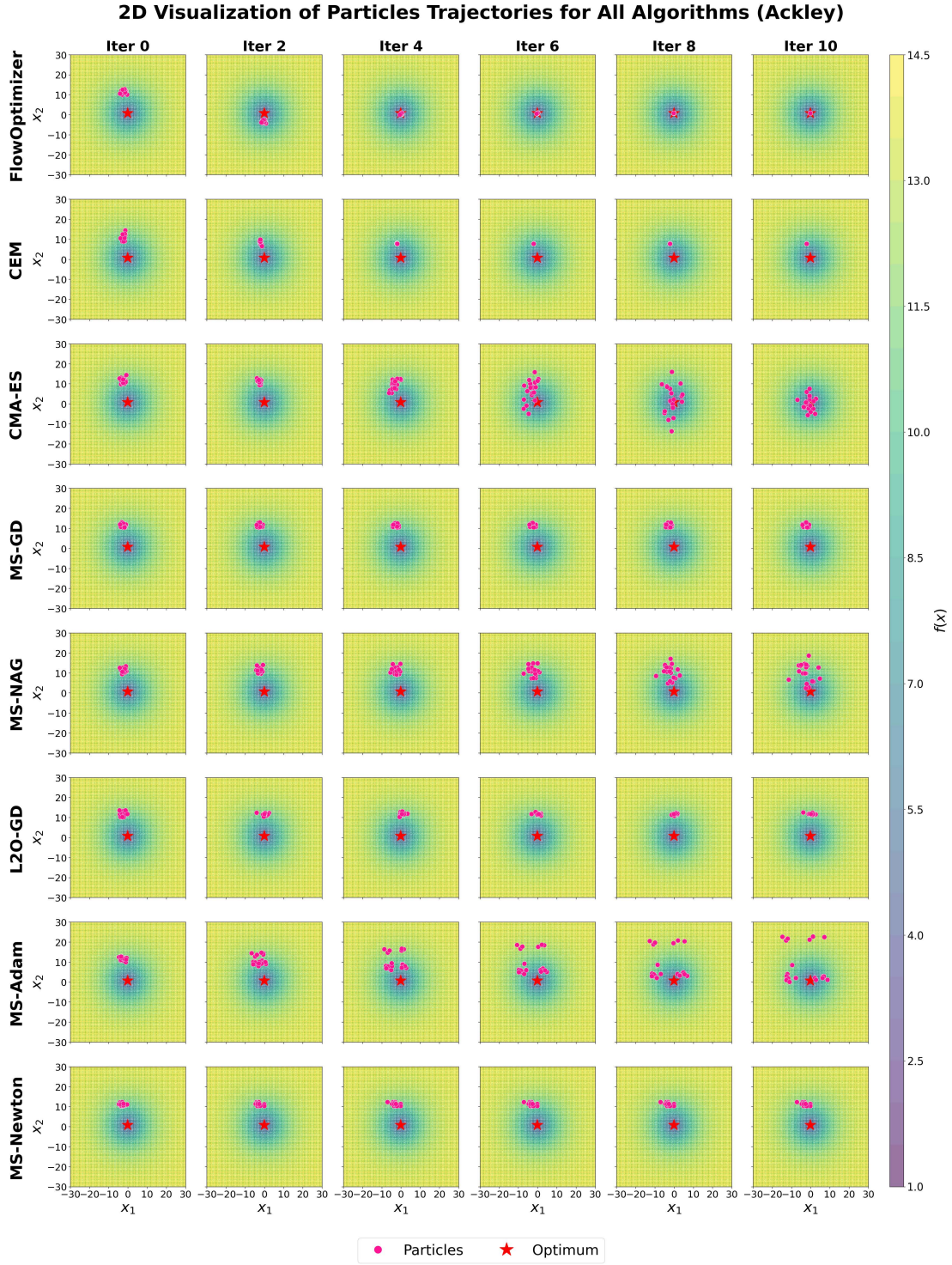


Figure 11. Visualization of Iteration Snapshots for All Methods - 2D Ackley Problem - Full Domain.

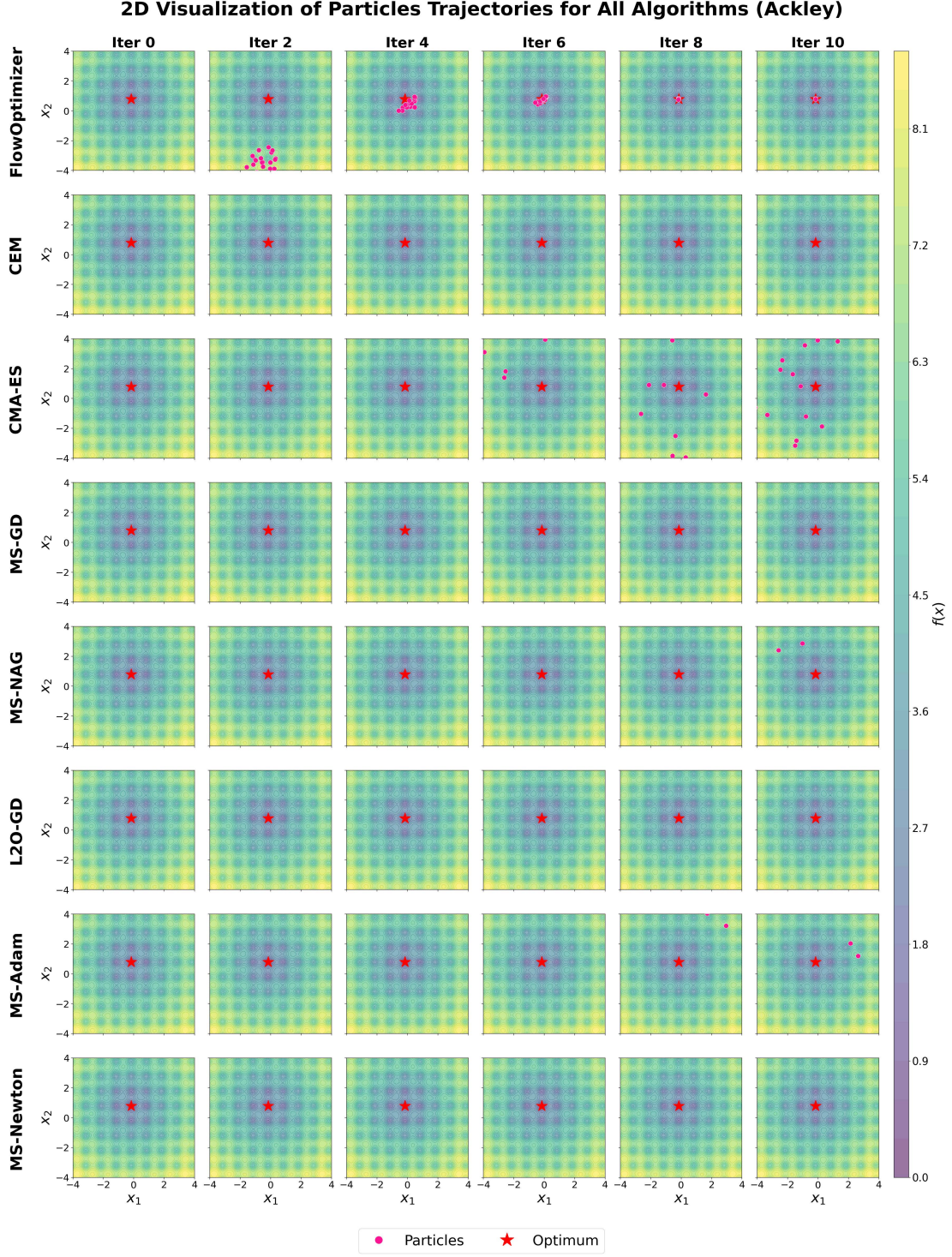


Figure 12. Visualization of Iteration Snapshots for All Methods - 2D Ackley Problem - Zoomed into $[-4, 4]^2$.

C.3. Ablation Studies

Next, we provide several ablation studies. In Table 2, we vary the parameter α . Our observations show that the best performing values are between 0.8 and 0.9 striking an optimal balance between prioritizing the best point of the population and the mean. Table 3 provides an ablation on the population size for the 20D standard optimization benchmarks. The results show that above some critical population size, FlowOptimizer can learn to navigate to high-quality solutions. Table 4 provides a comparison between using the Gaussian base sampler or not. Although our observations show that in general it is preferable to feed the population directly to the new iteration, we find that in some cases the additional sampling step might benefit exploration. In Table 5, we compare the performance of our method after pre-training and after fine-tuning across various dimensions. As expected, we observe that beyond some dimension, the pre-training phase becomes less effective which we attribute to its relatively naive sampling. Future work will study more sophisticated sampling schemes for training flow-based L2O frameworks. Table 6 shows the number of iterations, function evaluations and gradient evaluations for each method in the standard optimization benchmark experiments. Finally, Table 7 varies the training and evaluation dimensions for the dimension-agnostic experiments showing that if the dimension for training is too low, then performance will degrade as the evaluation dimension increases.

Table 2. Ablation on hyperparameter α in training loss term $\ell_k(\mathcal{X}^k) = \alpha \text{Best}(f(\mathcal{X}^k)) + (1 - \alpha) \text{Mean}(f(\mathcal{X}^k))$.

Problem Class		Hyperparameter α	0.0	0.2	0.4	0.6	0.8	0.9	1.0
Ackley 20D	Avg.	$ \text{Best}(f(\mathcal{X}^K)) - f^* / \text{Best}(f(\mathcal{X}^0)) - f^* $	0.23	4.8e-2	7.3e-3	4.9e-3	3.6e-3	4.3e-3	0.054
Rastrigin 20D	Avg.	$ \text{Best}(f(\mathcal{X}^K)) - f^* / \text{Best}(f(\mathcal{X}^0)) - f^* $	0.69	6.8e-3	5.8e-4	6.1e-4	3.0e-4	2.8e-4	0.72
Levy 20D	Avg.	$ \text{Best}(f(\mathcal{X}^K)) - f^* / \text{Best}(f(\mathcal{X}^0)) - f^* $	0.08	2.7e-3	8.1e-4	5.7e-4	2.5e-4	2.7e-3	6.3e-2

Table 3. Ablation on population size P . The average $|\text{Best}(f(\mathcal{X}^K)) - f^*| / |\text{Best}(f(\mathcal{X}^0)) - f^*|$ is reported in each case.

Problem Class	Population size P				
	5	10	20	30	50
Ackley 20D	8.2e-2	2.3e-2	3.6e-3	2.6e-3	1.1e-3
Rastrigin 20D	0.21	8.4e-2	3.0e-4	2.4e-4	1.7e-4
Levy 20D	9.3e-2	6.1e-2	2.5e-4	1.1e-4	7.6e-5

Table 4. Comparison on using the Gaussian base sampler. The average $|\text{Best}(f(\mathcal{X}^K)) - f^*| / |\text{Best}(f(\mathcal{X}^0)) - f^*|$ is reported.

Problem Class	Gaussian Base Sampler	Metric
Ackley 20D	Yes	5.9e-3
	No	3.6e-3
Rastrigin 20D	Yes	1.8e-4
	No	3.0e-4
Levy 20D	Yes	6.3e-3
	No	2.5e-4

Table 5. Performance and training efficiency comparison between pre-training and fine-tuning.

Problem Class	Metric	Pre-training	Fine-tuning
Ackley 2D	Number of epochs	100	2000
	Training time per epoch	0.2s	0.8s
	Avg. $ \text{Best}(f(\mathcal{X}^K)) - f^* / \text{Best}(f(\mathcal{X}^0)) - f^* $	0.05	1.5e-3
Ackley 5D	Number of epochs	100	2000
	Training time per epoch	0.2s	0.8s
	Avg. $ \text{Best}(f(\mathcal{X}^K)) - f^* / \text{Best}(f(\mathcal{X}^0)) - f^* $	0.08	2.8e-3
Ackley 10D	Number of epochs	100	2000
	Training time per epoch	0.3s	1.0s
	Avg. $ \text{Best}(f(\mathcal{X}^K)) - f^* / \text{Best}(f(\mathcal{X}^0)) - f^* $	0.12	3.1e-3
Ackley 20D	Number of epochs	100	2000
	Training time per epoch	0.4s	1.1s
	Avg. $ \text{Best}(f(\mathcal{X}^K)) - f^* / \text{Best}(f(\mathcal{X}^0)) - f^* $	0.17	3.6e-3
Ackley 50D	Number of epochs	100	2000
	Training time per epoch	0.4s	1.3s
	Avg. $ \text{Best}(f(\mathcal{X}^K)) - f^* / \text{Best}(f(\mathcal{X}^0)) - f^* $	0.47	9.9e-3

Table 6. Number of iterations, function evaluations and gradient evaluations for experiments in Fig. 4

Problem Class	Metric	RS	CEM	CMA-ES	MSGD	MSNAG	L2O-GD	FlowOptimizer
Ackley 20D	Iterations	78	57	54	73	72	73	10
	Function evals	1560	1140	1080	1460	1440	1460	200
	Gradient evals	-	-	-	1460	1440	1460	200
Rastrigin 20D	Iterations	165	103	93	163	160	163	10
	Function evals	3300	2060	1860	3260	3200	3260	200
	Gradient evals	-	-	-	3260	3200	3260	200
Levy 20D	Iterations	57	47	44	51	50	51	10
	Function evals	1140	940	880	1020	1000	1020	200
	Gradient evals	-	-	-	1020	1000	1020	200

 Table 7. Varying training and validation dimensions on Ackley. The average $|\text{Best}(f(\mathcal{X}^K)) - f^*| / |\text{Best}(f(\mathcal{X}^0)) - f^*|$ is reported.

Training dim.	Validation dim.			
	5	20	50	200
5	1.3e-2	0.76	1.89	2.74
20	7.9e-3	1.7e-2	7.4e-2	0.15

D. Additional Discussion

D.1. Discussion on Potential Performance Guarantees

Providing formal performance guarantees for L2O methods is inherently challenging, as the motivation behind L2O is to replace fixed algorithmic choices with learnable components, i.e., neural networks, in order to improve empirical performance (Amos et al., 2023). This added flexibility, however, often comes at the expense of classical algorithmic guarantees. Nevertheless, several research directions have emerged to formalize the behavior of L2O methods, including safeguarding mechanisms (Heaton et al., 2023), worst-case guarantees (Banert et al., 2024; Sambharya et al., 2025; Martin et al., 2025) and probabilistic generalization bounds (Saravanos et al., 2025; Sambharya & Stellato, 2025; Sucker et al., 2025).

We briefly discuss here a potential approach for establishing generalization bounds for FlowOptimizer. We highlight that the

same general framework used to derive PAC-Bayes generalization bounds on an optimization progress metric in (Saravanos et al., 2025) can be applied here, provided that a suitable performance metric is selected for the non-convex setting. A natural choice could be:

$$q = \min \left\{ \frac{f(x_{\text{best}}^K(\zeta; \theta)) - f^*(\zeta)}{f(x^0(\zeta)) - f^*(\zeta)}, 1 \right\} \quad (38)$$

which quantifies the final progress relative to initialization, where ζ and θ denote the problem instance and neural network parameters, respectively. Since this metric satisfies the criteria discussed in Section 5 of (Saravanos et al., 2025), the same argument can be used to obtain a generalization bound analogous to Theorem 3 in that work, after replacing their progress metric with (38). Following the same training and evaluation procedure would allow us to establish generalization bounds for FlowOptimizer. We note that such a bound would rely on either knowing the optimal objective value (not the optimal solution) or at least having a sufficiently accurate estimate of it. We leave the design and evaluation of more practical bounds that are suitable for non-convex optimization for future work.